Ide

# Last Man Standing

## On-Chain Survival Arena

**Monad Blitz Jogja — April 2026**

Dokumen Produk Hackathon

Versi 1.0 | Bahasa Indonesia

# Ringkasan Eksekutif

Last Man Standing adalah permainan eliminasi on-chain real-time di mana 20–50 pemain bergabung
dalam satu ruangan, dan setiap 10–15 detik, sejumlah pemain dieliminasi secara acak hingga hanya
satu yang tersisa sebagai pemenang.

Proyek ini menggabungkan randomness terverifikasi, gameplay real-time, dan prediction market
untuk penonton — menjadikannya demo yang kuat sekaligus produk yang layak secara bisnis.

## Pitch Singkat

Di era blockchain yang penuh dengan DeFi kompleks dan NFT statis, belum ada permainan on-chain yang benar-benar real-time dan melibatkan massa secara emosional. Last Man Standing mengisi celah ini dengan cara yang sederhana namun adiktif: lempar pemain ke dalam arena, biarkan randomness menentukan nasib mereka, dan hadirkan pengalaman menonton yang mendebarkan bagi siapa saja yang berada di luar arena.

Dibangun di atas Monad — blockchain dengan throughput tinggi dan latensi rendah — proyek ini membuktikan bahwa gaming on-chain bisa secepat dan seasyik gaming tradisional.

# Daftar Isi


1. Latar Belakang & Konteks Hackathon
2. Masalah & Peluang
3. Konsep Produk
4. Inovasi Inti
5. Arsitektur Sistem
6. Scope Hackathon: Apa yang Dibangun
7. Rencana Eksekusi 1 Hari
8. Strategi untuk Menang
9. Kekuatan & Kelemahan
10. Potensi Masa Depan

# 1. Latar Belakang & Konteks Hackathon

## Apa Itu Monad Blitz Jogja?

Monad Blitz Jogja adalah hackathon intensif 1 hari yang diselenggarakan oleh ekosistem Monad di Yogyakarta, April 2026. Event ini dirancang untuk mendorong builder lokal membangun aplikasi consumer-grade di atas Monad — blockchain EVM-compatible dengan performa jauh di atas Ethereum mainnet.

Monad diklaim mampu memproses ribuan transaksi per detik dengan finality yang sangat cepat, menjadikannya platform ideal untuk aplikasi yang membutuhkan respons real-time.

## Mengapa Hackathon Ini Penting?

- Monad sedang dalam fase ekspansi ekosistem dan aktif mencari proyek unggulan dari komunitas Asia Tenggara.
- Builder yang tampil menonjol berpotensi mendapat pendanaan lanjutan, inkubasi, atau kolaborasi ekosistem.
- Event ini menjadi ajang pembuktian bahwa Monad cocok untuk use case gaming dan consumer apps — bukan hanya DeFi.

## Area Fokus Utama

| Area | Relevansi untuk Proyek Ini |
|---|---|
| Games dengan randomness | Eliminasi acak berbasis VRF |
| SocialFi | Pengalaman komunal — menonton bersama, reaksi live |
| Prediction Markets | Penonton bertaruh pada siapa yang akan menang |
| Consumer Apps | UX sederhana, siapa pun bisa bermain langsung |

## Mengapa Ide Ini Cocok?

Last Man Standing menyentuh semua empat area fokus secara bersamaan. Ini bukan proyek niche — ini adalah demo yang bisa dinikmati oleh seluruh ruangan secara langsung saat presentasi.

## 2. Masalah & Peluang

### Masalah yang Diangkat

Gaming on-chain saat ini menghadapi dua masalah besar:

- Terlalu lambat. Blockchain konvensional tidak mampu mengeksekusi logika game real-time tanpa lag yang mengganggu pengalaman pemain.
- Terlalu kompleks untuk pemain kasual. Sebagian besar Web3 game membutuhkan pemahaman mendalam tentang wallet, gas, dan token — menghalangi adopsi massal.

### Mengapa Game Real-Time On-Chain Masih Langka?

- Latensi tinggi di Ethereum dan chain EVM standar membuat gameplay real-time hampir mustahil.
- Biaya gas yang tidak terprediksi menghancurkan monetisasi berbasis mikrotransaksi.
- Kurangnya randomness yang benar-benar terverifikasi membuat hasil permainan mudah dimanipulasi.

Monad menyelesaikan hambatan teknis ini. Yang tersisa adalah: siapa yang membangun game yang tepat?

### Peluang

- Pasar gaming blockchain diproyeksikan tumbuh signifikan dalam 2–3 tahun ke depan.
- Penonton (spectator economy) adalah segmen yang belum tersentuh di Web3 — Twitch dan YouTube membuktikan bahwa menonton orang bermain bisa sama menariknya dengan bermain sendiri.
- Prediction market berbasis gameplay adalah model bisnis baru yang belum banyak dijelajahi.

# 3. Konsep Produk

## Penjelasan Sederhana

> Bayangkan permainan kursi musik, tapi versi blockchain. Semua orang masuk ke ruangan virtual, membayar entry fee kecil, dan setiap beberapa detik — "kursi" berkurang.
> Siapa yang tidak dapat kursi, gugur. Satu pemain terakhir menang semua.

## Alur Permainan Lengkap

### 1. Bergabung (Join Phase)

- Pemain membuka dApp dan menghubungkan wallet.
- Memilih room aktif dan membayar entry fee (misalnya 0.01 MON).
- Room menampilkan daftar pemain yang telah bergabung secara real-time.
- Game dimulai ketika jumlah minimum pemain tercapai, atau timer lobby habis.

### 2. Hitung Mundur (Countdown)

- Countdown 30 detik sebelum ronde pertama.
- Penonton bisa mulai memasang prediksi siapa yang akan menang.
- Atmosfer membangun — layaknya momen sebelum pertandingan dimulai.

### 3. Ronde Eliminasi

- Setiap 10–15 detik, smart contract memanggil randomness engine.
- Sejumlah pemain (20–30% dari sisa pemain) dieliminasi secara acak.
- Pemain yang gugur melihat notifikasi eliminasi dan otomatis menjadi penonton.
- Proses berulang hingga tersisa 1 pemain.

### 4. Pemenang & Distribusi Hadiah

- Pemain terakhir dideklarasikan sebagai pemenang on-chain.
- Hadiah langsung ditransfer ke wallet pemenang secara otomatis.
- Hasil game tercatat permanen dan transparan di blockchain.

## Perjalanan Emosional Pengguna

| Fase | Emosi Pemain | Emosi Penonton |
|---|---|---|
| Lobby | Antisipasi | Kepo, mulai analisis |

| Fase | Emosi Pemain | Emosi Penonton |
|---|---|---|
| Ronde 1–2 | Lega / Kecewa | Tegang, mulai jagokan seseorang |
| Ronde tengah | Optimisme | Excited, prediksi semakin dalam |
| Ronde akhir | Adrenalin tinggi | Hampir seperti nonton final |
| Pengumuman | Euforia / Penerimaan | Kepuasan jika prediksi benar |

# 4. Inovasi Inti

## 4.1 Sistem Eliminasi On-Chain Real-Time

Berbeda dari game berbasis giliran (turn-based), Last Man Standing mengeksekusi logika eliminasi langsung di smart contract secara otomatis setiap interval waktu. Ini hanya mungkin dilakukan dengan throughput tinggi seperti yang ditawarkan Monad.

## 4.2 Randomness Terverifikasi (VRF)

Eliminasi tidak boleh bisa diprediksi atau dimanipulasi. Dua pendekatan tersedia:

- Versi Sederhana (Hackathon): Kombinasi block hash + timestamp + seed dari pemain sebagai pseudo-random. Cukup untuk demo dan aman untuk skala kecil.
- Versi Produksi: Integrasi Verifiable Random Function (VRF) — fungsi kriptografis yang menghasilkan angka acak yang dapat diverifikasi oleh siapapun bahwa prosesnya fair dan tidak dapat dimanipulasi.

## 4.3 Lapisan Strategi Pemain (Shield System)

Untuk menghindari persepsi "pure luck", tambahkan mekanisme shield:

- Setiap pemain mendapat 1 shield gratis per game.
- Shield bisa diaktifkan sekali untuk menghindari eliminasi di satu ronde.
- Penggunaan shield tercatat on-chain dan terlihat oleh penonton.
- Timing penggunaan shield menjadi elemen strategi: kapan momen yang tepat?

## 4.4 Lapisan Prediction Market

- Penonton yang tidak bermain bisa memasang taruhan pada pemain tertentu.
- Odds berubah dinamis mengikuti jumlah pemain yang tersisa.
- Jika prediksi benar, penonton mendapat hadiah dari pool prediksi.
- Ini menciptakan insentif finansial untuk menonton — bukan hanya hiburan pasif.

## 4.5 Keterlibatan Penonton (Spectator Engagement)

- Live leaderboard yang menampilkan siapa saja yang masih bertahan.
- Pemain yang gugur bisa mengirim emoji/reaksi ke arena.
- Feed aktivitas real-time: setiap eliminasi tercatat dan ditampilkan secara dramatis.
- Semua aktivitas ini mendorong sharing organik ke media sosial.

# 5. Arsitektur Sistem

## Smart Contract (On-Chain Logic)

Bertanggung jawab atas semua logika kritis:

- Manajemen room: buka, tutup, daftar pemain.
- Escrow entry fee dan distribusi hadiah otomatis.
- Eksekusi eliminasi per ronde menggunakan randomness.
- Pencatatan status game (siapa hidup, siapa gugur, ronde berapa).
- Manajemen shield per pemain.
- Pool prediksi penonton dan distribusi reward.

## Frontend (dApp)

Bertanggung jawab atas pengalaman pengguna:

- Koneksi wallet (MetaMask / WalletConnect).
- Tampilan lobby: daftar pemain, countdown timer, tombol join.
- Arena view: animasi eliminasi real-time, status pemain.
- Panel penonton: form prediksi, live feed, leaderboard.
- Notifikasi eliminasi yang dramatis dengan efek visual/audio.

## Backend Opsional (Automation Layer)

Untuk keperluan hackathon, backend ringan bisa digunakan untuk:

- Mengirim trigger transaksi eliminasi setiap interval waktu (sebagai "keeper").
- Menyediakan WebSocket untuk update real-time ke frontend.

Catatan: Dalam versi produksi, ini bisa digantikan oleh Chainlink Automation atau keeper network terdesentralisasi.

## Alur Randomness

```
Ronde N dimulai
  → Backend/Keeper memanggil fungsi eliminasi di smart contract
  → Smart contract mengambil block hash terbaru + seed
  → Menghasilkan daftar indeks pemain yang dieliminasi
  → Status pemain diupdate on-chain
  → Event dipancarkan → Frontend menangkap event → UI diupdate
```

## 6. Scope Hackathon: Apa yang Dibangun

### MUST HAVE — Fitur Wajib

- Smart contract: join room, entry fee, ronde eliminasi, distribusi pemenang.
- Randomness sederhana (block hash based) yang berfungsi.
- Frontend: halaman lobby, tampilan arena, notifikasi eliminasi.
- Koneksi wallet dan interaksi kontrak.
- Demo bisa dijalankan live dengan 5–10 wallet test.

### NICE TO HAVE — Jika Ada Waktu

- Shield system (1 shield per pemain).
- Panel penonton dengan prediksi sederhana.
- Animasi eliminasi yang dramatis.
- Live feed aktivitas game.

### DO NOT BUILD — Hindari Over-Engineering

- VRF sesungguhnya (terlalu kompleks untuk 1 hari).
- Sistem multi-room bersamaan.
- Token governance atau tokenomics kompleks.
- Mobile app native.
- Sistem leaderboard global antar game.

## 7. Rencana Eksekusi 1 Hari

### Pagi (08:00 – 12:00): Fondasi

| Waktu | Tugas |
|---|---|
| 08:00–09:00 | Finalisasi arsitektur, bagi tugas tim, setup repo & environment |
| 09:00–10:30 | Tulis dan deploy smart contract ke Monad testnet |
| 10:30–12:00 | Bangun halaman lobby: koneksi wallet, join room, daftar pemain |

✅ Target Pagi: Smart contract live di testnet, lobby bisa diakses.

### Siang (12:00 – 15:00): Core Game Loop

| Waktu | Tugas |
|---|---|
| 12:00–13:00 | Implementasi logika ronde eliminasi + randomness |
| 13:00–14:30 | Frontend arena view: animasi eliminasi, update status pemain real-time |
| 14:30–15:00 | Uji coba end-to-end dengan multiple wallet test |

✅ Target Siang: Satu game bisa dijalankan penuh dari join hingga pemenang.

### Sore (15:00 – 18:00): Polish & Fitur Tambahan

| Waktu | Tugas |
|---|---|
| 15:00–16:00 | UI polish: efek visual eliminasi, notifikasi dramatis |
| 16:00–17:00 | Tambah shield system atau panel penonton (jika waktu cukup) |
| 17:00–18:00 | Bug fixing, stress test, persiapan demo |

✅ Target Sore: Produk terasa "hidup" dan siap didemonstrasikan.

### Malam (18:00 – 21:00): Demo & Presentasi

| Waktu | Tugas |
|---|---|
| 18:00–19:00 | Latihan demo run: simulasi game penuh |
| 19:00–20:00 | Susun slide pitch: masalah, solusi, demo, visi |
| 20:00–21:00 | Final rehearsal, backup plan jika demo gagal |

✅ Target Malam: Tim percaya diri dan demo berjalan mulus.

# 8. Strategi untuk Menang

## Apa yang Dicari Juri?

| Kriteria | Yang Dinilai |
|---|---|
| Inovasi teknis | Apakah ada sesuatu yang baru secara teknis? |
| Relevansi ekosistem | Apakah memanfaatkan keunggulan Monad secara nyata? |
| Demo impact | Apakah produk terasa "nyata" dan menarik saat dipresentasikan? |
| Potensi bisnis | Apakah ini bisa jadi produk sungguhan? |

## Cara Memposisikan Ide Ini

> "Bukti nyata bahwa Monad bisa menghadirkan gaming real-time yang selama ini tidak mungkin dilakukan di blockchain lain."

Ini membuat proyek relevan secara langsung dengan kepentingan Monad sebagai platform, bukan hanya sebagai produk game biasa.

## Strategi Demo (PALING PENTING)

- Siapkan 5–10 wallet yang sudah pre-funded sebelum presentasi dimulai.
- Minta anggota tim atau relawan penonton untuk join game secara live dari device masing-masing.
- Jalankan game penuh di depan juri — biarkan mereka menyaksikan eliminasi terjadi secara real-time.
- Highlight kecepatan Monad: tunjukkan block time dan konfirmasi transaksi yang cepat secara eksplisit di layar.
- Ciptakan momen dramatis: saat 2 pemain tersisa, beri jeda, bangun suspense, lalu umumkan pemenang.

## Yang Perlu Ditekankan

- Monad Performance: Setiap eliminasi adalah transaksi on-chain. Di Ethereum, ini lambat dan mahal. Di Monad, terjadi dalam hitungan detik.
- Real-Time Experience: Tunjukkan bahwa blockchain bisa terasa seperti aplikasi Web2 dalam hal kecepatan dan responsivitas.
- Keterlibatan Penonton: Ajak juri untuk menjadi "penonton" dan merasakan excitement-nya secara langsung.

# 9. Kekuatan & Kelemahan

## Kekuatan

| Kekuatan | Penjelasan |
|---|---|
| Simplisitas konsep | Siapapun langsung paham cara bermainnya dalam 10 detik |
| Demo impact tinggi | Bisa dimainkan live di depan juri tanpa banyak penjelasan |
| Relevansi Monad | Secara langsung membuktikan keunggulan throughput Monad |
| Spektrum pengguna luas | Gamer, spekulan, dan penonton kasual semua bisa terlibat |
| Web3 native | Randomness terverifikasi, hadiah trustless, transparansi penuh |

## Kelemahan & Mitigasi

| Kelemahan | Risiko | Mitigasi |
|---|---|---|
| Terkesan luck-based | Pemain merasa tidak ada skill | Tambahkan shield system sebagai elemen keputusan |
| Kemiripan dengan gambling | Bisa ditolak di beberapa yurisdiksi | Framing sebagai "skill + strategy game", bukan pure gambling |
| Kedalaman gameplay terbatas | Sulit untuk retensi jangka panjang | Variasikan mekanisme per room: power-up, aliansi, dll. |
| Ketergantungan keeper | Jika keeper mati, game berhenti | Gunakan Chainlink Automation di versi produksi |

# 10. Potensi Masa Depan

## Evolusi Produk

Setelah hackathon, Last Man Standing bisa berkembang menjadi platform permainan on-chain yang lebih lengkap:

- Variasi Mode: Room dengan aturan berbeda — team mode, VIP room, tournament bracket.
- NFT Integration: Karakter NFT dengan buff tertentu (lebih banyak shield, odds lebih rendah dieliminasi).
- Seasonal Tournament: Liga bulanan dengan hadiah besar dari sponsor ekosistem.
- Cross-Chain Expansion: Deploy ke chain EVM lain yang kompatibel dengan Monad.

## Model Monetisasi

| Sumber Pendapatan | Mekanisme |
|---|---|
| Platform fee | 5–10% dari setiap pool hadiah |
| Prediction market spread | Margin dari odds prediksi penonton |
| VIP room | Entry fee lebih tinggi, hadiah lebih besar |
| Iklan / Sponsorship | Brand bisa "mensponsori" room tertentu |
| NFT sales | Karakter atau item eksklusif |

## Visi Platform

Last Man Standing bukan sekadar satu game — ini adalah infrastruktur untuk permainan eliminasi on-chain yang bisa di-fork, dikustomisasi, dan diintegrasikan oleh developer lain. Bayangkan Roblox versi on-chain, di mana siapapun bisa membuat "survival room" dengan aturan mereka sendiri.

*— Dokumen ini disiapkan untuk Monad Blitz Jogja, April 2026 —*

# Pembagian job

# Last Man Standing

## Technical Execution Guide

**Monad Blitz Jogja — 25 April 2026**

Mulai: 22 April 2026 | Deadline: 25 April 2026

Tim: 3 Orang | Smart Contract + Frontend + Backend

| 🔗 Smart Contract | 🖥️ Frontend | ⚙️ Backend |
|---|---|---|

# Overview & Pembagian Tim

## Struktur Tim

| Role | Siapa | Domain Utama | Tools |
|---|---|---|---|
| Smart Contract | SC Dev | Solidity, Monad testnet, deploy | Foundry, Solidity ^0.8 |
| Frontend | FE Dev | React dApp, wallet, UI real-time | Next.js, wagmi, ethers.js |
| Backend | BE Dev | Game keeper, WebSocket, timer | Node.js, Express, Socket.io |

## Tech Stack Keseluruhan

| Layer | Teknologi | Keterangan |
|---|---|---|
| Blockchain | Monad Testnet | EVM-compatible, throughput tinggi |
| Smart Contract | Solidity ^0.8.20 | Game logic, randomness, prize pool |
| SC Framework | Foundry | Compile, test, deploy |
| Frontend Framework | Next.js 14 + TypeScript | React, routing, SSR optional |
| Wallet / Web3 | wagmi v2 + ethers.js v6 | Koneksi wallet, baca/tulis kontrak |
| UI | Tailwind CSS + shadcn/ui | Styling cepat dan konsisten |
| Backend | Node.js + Express | REST API ringan + WebSocket |
| Realtime | Socket.io | Push event ke semua client |
| State (BE) | In-memory / Redis (opsional) | Game state sementara |
| Deploy SC | Monad Testnet RPC | Via Hardhat deploy script |

## Prinsip Eksekusi 2 Hari

- Hari 1 (22–23 Apr): Semua infrastruktur + core game loop harus jalan.
- Hari 2 (23–25 Apr): Polish, opsional features, demo preparation.
- Wajib selesai Hari 1: SC deploy, BE running, FE bisa join + lihat eliminasi.
- Jangan mulai fitur opsional sebelum wajib 100% berjalan.

# Master Timeline 2 Hari

## Hari 1 — 22 April (Foundation Day)

| Waktu | SC | Frontend | Backend |
|---|---|---|---|
| 08:00–09:00 | Kick-off: finalize struktur kontrak, bagi repo | Setup Next.js + wagmi + Tailwind | Setup Node.js + Express + Socket.io |
| 09:00–11:00 | Tulis GameRoom.sol: join, state, event | Halaman Lobby: wallet connect, room list | Endpoint /rooms, /join; WebSocket setup |
| 11:00–13:00 | Tulis logika eliminasi + randomness + prize | Halaman Arena: daftar pemain live | Keeper loop: trigger eliminasi tiap N detik |
| 13:00–14:00 | Deploy ke Monad testnet, share ABI | Integrasi kontrak: join room on-chain | Broadcast event eliminasi via Socket.io |
| 14:00–16:00 | Testing kontrak (Hardhat test / manual) | Tampilkan eliminasi real-time di UI | Sync state BE dengan event on-chain |
| 16:00–18:00 | Fix bugs SC, bantu FE dengan ABI/events | Winner screen + notifikasi eliminasi | Stabilkan keeper, handle edge case |
| 18:00–20:00 | End-to-end test full game loop | End-to-end test full game loop | End-to-end test full game loop |

## Hari 2 — 23–25 April (Polish + Demo Day)

| Waktu | SC (Lead) | Frontend | Backend |
|---|---|---|---|
| 08:00–10:00 | Shield system on-chain (wajib opsional) | Shield activation button di UI | Broadcast shield event ke penonton |
| 10:00–12:00 | Prediction pool contract (opsional) | Panel penonton + prediksi (opsional) | API prediksi + distribusi reward |
| 12:00–14:00 | Security review kontrak, fix issues | UI polish: animasi, sound, efek dramatis | Stress test, handle reconnect Socket |
| 14:00–16:00 | Prepare demo: pre-fund 10 test wallets | Rehearsal demo dengan tim | Pastikan server stabil untuk demo |
| 16:00–18:00 | Buffer: fix apapun yang rusak | Buffer: fix apapun yang rusak | Buffer: fix apapun yang rusak |
| 18:00–25 Apr | Persiapan pitch & presentasi | Screenshot / video demo backup | Deploy ke server publik (opsional) |

# Smart Contract — Detail Teknis

**👤 Owner: Smart Contract**

## Struktur File

```
contracts/
├── GameRoom.sol          ← Kontrak utama (WAJIB)
├── GameFactory.sol       ← Factory buat room (WAJIB)
├── IRandomness.sol       ← Interface randomness (WAJIB)
├── ShieldSystem.sol      ← Shield mechanic (OPSIONAL)
└── PredictionPool.sol    ← Prediction market (OPSIONAL)

scripts/
├── deploy.js             ← Deploy script
└── seed.js               ← Pre-fund test wallets

test/
└── GameRoom.test.js      ← Unit tests
```

## GameRoom.sol — Spesifikasi Lengkap

### State Variables (Wajib)

```
// Room configuration
uint256 public roomId;
uint256 public entryFee;          // dalam wei (e.g., 0.01 MON)
uint256 public minPlayers;        // minimal pemain (e.g., 5)
uint256 public maxPlayers;        // maksimal pemain (e.g., 50)
uint256 public eliminationPct;    // % dieliminasi per ronde (e.g., 25)
uint256 public roundInterval;     // detik antar ronde (e.g., 12)

// Game state
enum GameStatus { WAITING, COUNTDOWN, ACTIVE, FINISHED }
GameStatus public status;
address[] public players;         // daftar semua pemain
mapping(address => bool) public isAlive;
mapping(address => bool) public hasJoined;
address public winner;
uint256 public currentRound;
uint256 public lastRoundTime;

// Prize pool
uint256 public prizePool;
uint256 public platformFeePct;    // e.g., 5
address public treasury;
```

### Functions Wajib

| Function | Visibility | Keterangan |
|---|---|---|
| joinRoom() | external payable | Pemain join + bayar entry fee. Tambah ke players[], set isAlive=true |
| startGame() | external (keeper/owner) | Ubah status ke ACTIVE, set lastRoundTime |
| executeRound() | external (keeper) | Jalankan 1 ronde eliminasi. Panggil _getRandomSeed(), pilih yang dieliminasi |
| _eliminatePlayers(uint 256 seed) | internal | Logika eliminasi: shuffle indeks, hapus % pertama |
| claimPrize() | external | Pemenang claim hadiah. Transfer prizePool - platformFee |
| getAlivePlayers() | external view | Return array address pemain yang masih hidup |
| getGameStatus() | external view | Return status, currentRound, jumlah hidup |

### Randomness — Implementasi Hackathon

```
function _getRandomSeed(uint256 roundNum) internal view returns (uint256) {
    return uint256(keccak256(abi.encodePacked(
        block.timestamp,
        block.prevrandao,   // EIP-4399, lebih baik dari blockhash
        msg.sender,
        roundNum,
        players.length
    )));
}

// Untuk produksi: ganti dengan Chainlink VRF v2.5
// Untuk hackathon: prevrandao cukup aman dan tidak bisa diprediksi user biasa
```

### Logika Eliminasi — Implementasi

```
function _eliminatePlayers(uint256 seed) internal {
    address[] memory alive = getAlivePlayers();
    uint256 elimCount = (alive.length * eliminationPct) / 100;
    if (elimCount == 0) elimCount = 1; // minimal 1 dieliminasi

    // Fisher-Yates partial shuffle
    for (uint256 i = 0; i < elimCount; i++) {
        uint256 j = i + (uint256(keccak256(abi.encodePacked(seed, i))) %
(alive.length - i));
        address tmp = alive[i];
        alive[i] = alive[j];
        alive[j] = tmp;
```

```
        isAlive[alive[i]] = false;
        emit PlayerEliminated(alive[i], currentRound);
    }

    // Cek kondisi menang
    address[] memory remaining = getAlivePlayers();
    if (remaining.length == 1) {
        winner = remaining[0];
        status = GameStatus.FINISHED;
        emit GameWinner(winner, prizePool);
    }
}
```

### Events yang Harus Dideklarasikan

```
event PlayerJoined(address indexed player, uint256 totalPlayers);
event GameStarted(uint256 timestamp, uint256 totalPlayers);
event RoundStarted(uint256 roundNumber, uint256 playersAlive);
event PlayerEliminated(address indexed player, uint256 round);
event GameWinner(address indexed winner, uint256 prize);
event PrizeClaimed(address indexed winner, uint256 amount);
event ShieldActivated(address indexed player, uint256 round); // opsional
```

## GameFactory.sol — Spesifikasi

Factory dipakai untuk membuat banyak room game secara dinamis. Frontend bisa list semua room yang aktif.

```
contract GameFactory {
    address[] public allRooms;
    mapping(address => bool) public isValidRoom;

    event RoomCreated(address roomAddress, uint256 entryFee, address creator);

    function createRoom(
        uint256 _entryFee,
        uint256 _minPlayers,
        uint256 _maxPlayers,
        uint256 _eliminationPct,
        uint256 _roundInterval
    ) external returns (address) {
        GameRoom room = new GameRoom(...args, msg.sender);
        allRooms.push(address(room));
        isValidRoom[address(room)] = true;
        emit RoomCreated(address(room), _entryFee, msg.sender);
        return address(room);
    }
```

```
    function getActiveRooms() external view returns (address[] memory) { ... }
}
```

## Deploy & Setup

### Langkah Deploy

- Install dependency: npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox
- Config hardhat.config.js dengan Monad testnet RPC
- Tulis deploy script, run: npx hardhat run scripts/deploy.js --network monad
- Copy ABI dari artifacts/ ke folder shared/ yang bisa diakses FE dan BE
- Share contract address ke seluruh tim via .env atau config bersama

```
// hardhat.config.js
networks: {
  monad: {
    url: "https://testnet-rpc.monad.xyz",  // cek docs Monad terbaru
    accounts: [process.env.PRIVATE_KEY],
    chainId: 10143  // sesuaikan dengan chainId Monad testnet
  }
}
```

## Output SC yang Harus Diserahkan ke Tim

| Output | Format | Diserahkan ke | Kapan |
|---|---|---|---|
| Contract Address | .env / config.ts | FE + BE | Setelah deploy Hari 1 |
| ABI GameRoom | gameRoom.abi.json | FE + BE | Setelah deploy Hari 1 |
| ABI GameFactory | gameFactory.abi.json | FE + BE | Setelah deploy Hari 1 |
| Event signatures | Dokumentasi text | BE (listener) | Hari 1 pagi |
| Test wallet addresses | Daftar address + key | FE (demo) | Hari 2 |

# Backend — Detail Teknis

**👤 Owner: Backend Developer**

## Peran Backend dalam Sistem

Smart contract TIDAK bisa menjalankan dirinya sendiri secara otomatis.
Backend berperan sebagai "Keeper" — yang memicu eksekusi ronde setiap N detik.
Backend juga menjadi jembatan real-time antara blockchain dan frontend via WebSocket.

## Struktur File

```
backend/
├── src/
│   ├── index.js              ← Entry point, start server
│   ├── config.js             ← Env vars, contract address, ABI
│   ├── keeper/
│   │   ├── GameKeeper.js     ← Core: trigger eliminasi on-chain (WAJIB)
│   │   └── keeperManager.js  ← Manage multiple room keepers
│   ├── socket/
│   │   └── socketHandler.js  ← Socket.io event handling (WAJIB)
│   ├── chain/
│   │   └── contractListener.js ← Listen event dari kontrak (WAJIB)
│   └── routes/
│       └── rooms.js          ← REST API endpoints
├── shared/
│   ├── gameRoom.abi.json     ← ABI dari SC (di-share)
│   └── gameFactory.abi.json
├── .env
└── package.json
```

## GameKeeper.js — Inti Backend (WAJIB)

Keeper adalah loop yang secara berkala memanggil executeRound() di smart contract.

```
const { ethers } = require("ethers");

class GameKeeper {
  constructor(roomAddress, provider, signer, io) {
    this.contract = new ethers.Contract(roomAddress, ROOM_ABI, signer);
    this.io = io;  // Socket.io instance
    this.interval = null;
    this.roomAddress = roomAddress;
  }
```

```
  async start(intervalMs = 12000) {
    console.log(`Keeper started for room ${this.roomAddress}`);
    this.interval = setInterval(async () => {
      await this.triggerRound();
    }, intervalMs);
  }

  async triggerRound() {
    try {
      const status = await this.contract.status();
      if (status !== 2) return; // 2 = ACTIVE

      const tx = await this.contract.executeRound();
      await tx.wait();
      console.log("Round executed, tx:", tx.hash);
    } catch (err) {
      console.error("Keeper error:", err.message);
    }
  }

  stop() { clearInterval(this.interval); }
}
```

## contractListener.js — Event Listener (WAJIB)

Listen semua event dari kontrak dan broadcast ke frontend via Socket.io.

```
function setupContractListeners(contract, io) {

  contract.on("PlayerJoined", (player, totalPlayers) => {
    io.emit("player_joined", {
      player,
      totalPlayers: totalPlayers.toString(),
      timestamp: Date.now()
    });
  });

  contract.on("PlayerEliminated", (player, round) => {
    io.emit("player_eliminated", {
      player,
      round: round.toString(),
      timestamp: Date.now()
    });
  });

  contract.on("RoundStarted", (roundNumber, playersAlive) => {
    io.emit("round_started", {
      round: roundNumber.toString(),
      playersAlive: playersAlive.toString()
```

```
    });
  });

  contract.on("GameWinner", (winner, prize) => {
    io.emit("game_winner", {
      winner,
      prize: ethers.formatEther(prize) + " MON"
    });
  });
}
```

## REST API Endpoints (WAJIB)

| Method | Endpoint | Response | Kegunaan |
|---|---|---|---|
| GET | /api/rooms | Array room aktif + status | FE list semua room |
| GET | /api/rooms/:address | Detail room + players | FE room detail |
| POST | /api/rooms/start/:address | { success } | Trigger game start (admin/keeper) |
| GET | /api/rooms/:address/players | Array address + alive status | FE update player list |

## Socket.io Events — Kontrak dengan Frontend

| Event Name | Arah | Payload | Keterangan |
|---|---|---|---|
| player_joined | BE → FE | { player, totalPlayers } | Ada pemain baru join |
| player_eliminated | BE → FE | { player, round } | Pemain dieliminasi |
| round_started | BE → FE | { round, playersAlive } | Ronde baru dimulai |
| game_winner | BE → FE | { winner, prize } | Game selesai, ada pemenang |
| game_status | BE → FE | { status, round, alive } | Update berkala (tiap 5 detik) |
| spectator_react | FE → BE → FE | { emoji, from } | Penonton kirim reaksi (opsional) |

## Output BE yang Harus Diserahkan ke FE

| Output | Format | Kapan |
|---|---|---|
| Base URL server | http://localhost:3001 atau URL publik | Hari 1 pagi (segera) |

| Output | Format | Kapan |
|---|---|---|
| Socket event names + payload | Dokumen / comment di kode | Hari 1 sebelum integrasi |
| REST API endpoints | Postman collection / README | Hari 1 siang |
| WebSocket connection example | Snippet JS | Hari 1 siang |

# Frontend — Detail Teknis

👤 **Owner: Frontend Developer**

## Struktur File

```
frontend/src/
├── pages/ (atau app/ untuk Next.js 14)
│   ├── index.tsx              ← Landing / room list
│   ├── room/[address].tsx     ← Arena page (CORE)
│   └── spectate/[address].tsx ← Spectator view (opsional)
├── components/
│   ├── WalletConnect.tsx      ← Connect wallet button (WAJIB)
│   ├── RoomCard.tsx           ← Card info room (WAJIB)
│   ├── PlayerList.tsx         ← Daftar pemain + status (WAJIB)
│   ├── EliminationFeed.tsx    ← Feed event real-time (WAJIB)
│   ├── CountdownTimer.tsx     ← Timer hitung mundur (WAJIB)
│   ├── WinnerScreen.tsx       ← Layar pemenang (WAJIB)
│   ├── ShieldButton.tsx       ← Aktifkan shield (OPSIONAL)
│   └── SpectatorPanel.tsx     ← Panel prediksi penonton (OPSIONAL)
├── hooks/
│   ├── useGameRoom.ts         ← Hook baca state kontrak
│   ├── useSocket.ts           ← Hook Socket.io
│   └── useWallet.ts           ← Hook wallet state
├── lib/
│   ├── contracts.ts           ← ABI + address konstanta
│   └── socket.ts              ← Socket.io client instance
└── styles/
    └── globals.css
```

## Halaman & Komponen — Detail per Halaman

### 1. Halaman Utama / Lobby (index.tsx) — WAJIB

| Elemen UI | Data Source | Aksi |
|---|---|---|
| Header + Connect Wallet | wagmi useAccount | Trigger connect modal |
| List Room Aktif | GET /api/rooms | Klik → masuk room detail |
| Room Card: fee, pemain, status | API + socket update | Join atau Spectate |
| Tombol "Buat Room" (opsional) | GameFactory contract | Open form, deploy room baru |

### 2. Halaman Arena / Room (room/[address].tsx) — WAJIB (INTI)

| Elemen UI | Data Source | Aksi / Update Trigger |
|---|---|---|
| Status bar: Ronde, Alive count | Socket: round_started | Auto-update tiap ronde |
| PlayerList: grid avatar + status | Socket: player_eliminated | Pemain gugur → gray/cross effect |
| CountdownTimer | Socket: round_started | Hitung mundur ke ronde berikutnya |
| EliminationFeed | Socket: player_eliminated | Append item baru (sliding feed) |
| Tombol JOIN (jika WAITING) | GameRoom.joinRoom() | Bayar entry fee, trigger tx |
| Status "KAMU GUGUR" | useAccount + isAlive | Muncul jika player dieliminasi |
| WinnerScreen overlay | Socket: game_winner | Muncul fullscreen saat game selesai |

### 3. Hook: useSocket.ts — WAJIB

```
import { useEffect, useState } from "react";
import { io } from "socket.io-client";

const socket = io(process.env.NEXT_PUBLIC_BACKEND_URL!);

export function useSocket(roomAddress: string) {
  const [eliminated, setEliminated] = useState<string[]>([]);
  const [round, setRound] = useState(0);
  const [winner, setWinner] = useState<string | null>(null);

  useEffect(() => {
    socket.on("player_eliminated", (data) => {
      setEliminated(prev => [...prev, data.player]);
    });
    socket.on("round_started", (data) => {
      setRound(Number(data.round));
    });
    socket.on("game_winner", (data) => {
      setWinner(data.winner);
    });
    return () => { socket.off("player_eliminated"); socket.off("round_started");
socket.off("game_winner"); };
  }, []);

  return { eliminated, round, winner };
}
```

### 4. Hook: useGameRoom.ts — WAJIB

```
import { useContractRead } from "wagmi";
import { GAME_ROOM_ABI, getContractConfig } from "@/lib/contracts";

export function useGameRoom(address: `0x${string}`) {
  const { data: alivePlayers } = useContractRead({
    ...getContractConfig(address),
    functionName: "getAlivePlayers",
    watch: true,
  });

  const { data: gameStatus } = useContractRead({
    ...getContractConfig(address),
    functionName: "getGameStatus",
    watch: true,
  });

  return { alivePlayers, gameStatus };
}
```

### 5. Interaksi Write ke Kontrak — Join Room

```
import { useContractWrite, usePrepareContractWrite } from "wagmi";
import { parseEther } from "ethers";

function JoinButton({ roomAddress, entryFee }) {
  const { config } = usePrepareContractWrite({
    address: roomAddress,
    abi: GAME_ROOM_ABI,
    functionName: "joinRoom",
    value: parseEther(entryFee),
  });

  const { write, isLoading } = useContractWrite(config);

  return (
    <button onClick={() => write?.()} disabled={isLoading}>
      {isLoading ? "Joining..." : `Join — ${entryFee} MON`}
    </button>
  );
}
```

## UX Flow — Prioritas Visual

| State Game | Yang Ditampilkan | Prioritas |
|---|---|---|
| WAITING | Lobby: list pemain, tombol JOIN, countdown ke start | WAJIB |

| State Game | Yang Ditampilkan | Prioritas |
|---|---|---|
| ACTIVE - Hidup | Arena: player grid, feed, countdown ronde, badge "HIDUP" | WAJIB |
| ACTIVE - Gugur | Arena sama + overlay merah "KAMU GUGUR", mode spectator | WAJIB |
| FINISHED - Menang | WinnerScreen besar, konfeti, prize amount, tombol claim | WAJIB |
| FINISHED - Kalah | Layar "Kamu tidak menang", link ke room baru | WAJIB |
| Spectator Mode | Arena view tanpa tombol join, ada panel prediksi | OPSIONAL |

## Checklist Output FE

| Output | Status Target | Hari |
|---|---|---|
| Wallet connect berjalan di Monad testnet | WAJIB selesai | Hari 1 pagi |
| Lobby menampilkan room dari API | WAJIB selesai | Hari 1 siang |
| Join room + bayar on-chain berjalan | WAJIB selesai | Hari 1 siang |
| Arena real-time update via Socket | WAJIB selesai | Hari 1 sore |
| Eliminasi tampil dramatis di UI | WAJIB selesai | Hari 1 sore |
| Winner screen muncul otomatis | WAJIB selesai | Hari 1 malam |
| UI polish + animasi halus | OPSIONAL | Hari 2 |
| Spectator panel + prediksi | OPSIONAL | Hari 2 |

# Fitur: Wajib vs Opsional

## 🔴 WAJIB — Harus Selesai Hari 1

| Fitur | SC | BE | FE | Estimasi |
|---|---|---|---|---|
| Deploy kontrak ke Monad testnet | ✅ | - | - | 2 jam |
| joinRoom() + entry fee on-chain | ✅ | - | ✅ | 2 jam |
| Logika eliminasi per ronde | ✅ | - | - | 2 jam |
| Keeper: trigger ronde otomatis | - | ✅ | - | 2 jam |
| Event listener + Socket.io broadcast | - | ✅ | - | 2 jam |
| Lobby: list room + tombol join | - | ✅ | ✅ | 2 jam |
| Arena: player grid + feed eliminasi | - | - | ✅ | 3 jam |
| Winner screen + distribusi hadiah | ✅ | ✅ | ✅ | 2 jam |
| Koneksi wallet (MetaMask/Monad) | - | - | ✅ | 1 jam |

## 🟡 OPSIONAL — Dikerjakan Hari 2 Jika Wajib Sudah Selesai

| Fitur | SC | BE | FE | Estimasi |
|---|---|---|---|---|
| Shield system (1 per pemain) | ✅ | ✅ | ✅ | 3 jam |
| Prediction market (penonton bet) | ✅ | ✅ | ✅ | 4 jam |
| Animasi konfeti + sound effect | - | - | ✅ | 1 jam |
| Spectator mode (view only) | - | ✅ | ✅ | 2 jam |
| Room creation dari UI | ✅ | - | ✅ | 2 jam |
| Live emoji reactions dari penonton | - | ✅ | ✅ | 1 jam |

## ⛔ JANGAN DIBANGUN — Terlalu Kompleks untuk 2 Hari

- VRF on-chain sesungguhnya (Chainlink VRF) — setup terlalu lama, simpan untuk post-hackathon.
- Multi-room bersamaan dengan load balancing — terlalu kompleks.
- Token governance atau DAO mechanism.
- Mobile app native (React Native / Flutter).
- Sistem anti-cheat kompleks atau zk-proof.
- Database permanen (PostgreSQL/MongoDB) — pakai in-memory untuk hackathon.

# Integrasi & Kontrak Antar Tim

## Dependency Map — Siapa Butuh Apa dari Siapa

| Pihak | Butuh dari SC | Butuh dari BE | Memberikan ke |
|---|---|---|---|
| SC (Lead) | - | - | ABI, address, events ke FE + BE |
| Backend | ABI + address (untuk keeper & listener) | - | Socket events + REST API ke FE |
| Frontend | ABI + address (untuk write tx) | Socket events + REST API | Demo UI ke semua |

## Urutan Handoff yang Benar

URUTAN WAJIB:

1. SC: Deploy kontrak → share address + ABI ke FE dan BE (target: Hari 1, jam 13:00)
2. BE: Setup listener dengan ABI → mulai broadcast Socket (target: Hari 1, jam 14:00)
3. FE: Integrasi ABI untuk write tx + connect ke Socket (target: Hari 1, jam 15:00)
4. Semua: End-to-end test bersama (target: Hari 1, jam 17:00)

JANGAN: FE dan BE menunggu SC selesai sempurna.
BOLEH: FE dan BE pakai ABI dummy/mock selagi SC masih dikerjakan.

## Shared Config — .env Template

```
# .env.shared — bagikan ke seluruh tim

# Monad Testnet
MONAD_RPC_URL=https://testnet-rpc.monad.xyz
CHAIN_ID=10143

# Contract Addresses (diisi SC setelah deploy)
GAME_FACTORY_ADDRESS=0x...
GAME_ROOM_IMPLEMENTATION=0x...

# Backend
BACKEND_URL=http://localhost:3001
BACKEND_PORT=3001

# Frontend
NEXT_PUBLIC_BACKEND_URL=http://localhost:3001
```

```
NEXT_PUBLIC_CHAIN_ID=10143

# SC Deploy (JANGAN SHARE private key ke FE/BE)
PRIVATE_KEY=0x... (hanya di SC .env)
KEEPER_PRIVATE_KEY=0x... (hanya di BE .env)
```

## Checklist Integrasi — Sebelum End-to-End Test

| Checkpoint | Dicek oleh | Status |
|---|---|---|
| ABI sudah di-copy ke FE dan BE | SC Lead | [ ] |
| Contract address sudah di .env FE dan BE | SC Lead | [ ] |
| BE bisa connect ke Monad RPC | BE Dev | [ ] |
| BE bisa listen event PlayerJoined | BE Dev | [ ] |
| FE bisa connect wallet ke Monad testnet | FE Dev | [ ] |
| FE bisa call joinRoom() dan tunggu tx confirmed | FE Dev | [ ] |
| FE menerima Socket event player_eliminated | FE Dev | [ ] |
| Full game loop dari join → eliminasi → winner | Semua | [ ] |

# Persiapan Demo & Presentasi

## Setup Demo — H-1 Sebelum Hackathon (24 April)

- Buat 8–10 test wallet, fund masing-masing dengan MON testnet dari faucet.
- Deploy ulang kontrak fresh (clean state) ke Monad testnet.
- Test full game loop sekali dengan semua wallet tersebut.
- Simpan semua private key test wallet dalam file terpisah (JANGAN di repo publik).
- Siapkan browser window terpisah untuk setiap "pemain" yang akan demo.
- Siapkan satu window lagi sebagai "spectator view" untuk ditampilkan ke juri.

## Skrip Demo (Run Selama Presentasi)

| Langkah | Siapa yang Melakukan | Apa yang Ditampilkan ke Juri |
|---|---|---|
| 1. Buka lobby | FE Dev (screen share) | List room aktif, tombol connect wallet |
| 2. Connect 3–5 wallet | Tim + sukarelawan dari audience | Setiap wallet join → muncul di lobby |
| 3. Semua join room | SC Lead narasi | Tx on-chain cepat, konfirmasi Monad |
| 4. Game start, hitung mundur | BE Dev running keeper | Timer countdown dramatis |
| 5. Ronde 1: eliminasi terjadi | Otomatis | Nama merah di grid, feed muncul |
| 6. Ronde 2–3: semakin sedikit | Otomatis | Tension naik, audience excited |
| 7. Final 2 pemain tersisa | SC Lead bangun hype | Big moment, countdown terakhir |
| 8. Pemenang diumumkan | Otomatis | Winner screen, prize amount on-chain |
| 9. Klaim hadiah | Pemain menang klik claim | Tx on-chain, balance bertambah |

## Backup Plan

- Jika Monad testnet lambat: siapkan video recording demo yang berhasil sebelumnya.
- Jika Socket.io disconnect: refresh halaman, reconnect otomatis.
- Jika wallet tidak mau connect: gunakan wallet yang sudah di-authorize sebelumnya.
- Jika keeper gagal: SC Lead bisa trigger executeRound() manual dari Hardhat console.

## Poin yang Wajib Disebutkan ke Juri

1. "Setiap eliminasi adalah transaksi on-chain yang dikonfirmasi oleh Monad dalam hitungan detik."
2. "Randomness menggunakan prevrandao — tidak bisa dimanipulasi oleh pemain maupun operator."
3. "Prize pool 100% trustless — kontrak mendistribusikan otomatis, tidak ada yang bisa ambil."
4. "Ini menyentuh 4 area fokus Monad: Games, SocialFi, Prediction, Consumer App."
5. "Dibangun dalam 2 hari — visi produknya jauh lebih besar, ini hanya MVP-nya."

# Quick Reference — Cheatsheet Tim

## Commands Penting

| Konteks | Command | Keterangan |
|---|---|---|
| SC | npx hardhat compile | Compile semua kontrak |
| SC | npx hardhat test | Jalankan unit tests |
| SC | npx hardhat run scripts/deploy.js --network monad | Deploy ke testnet |
| SC | npx hardhat console --network monad | Debug interaktif |
| BE | npm run dev | Start server development |
| BE | node src/index.js | Start server production |
| FE | npm run dev | Start Next.js dev server |
| FE | npm run build && npm start | Production build |

## Port & URL Default

| Service | URL / Port | Catatan |
|---|---|---|
| Frontend | http://localhost:3000 | Next.js dev server |
| Backend | http://localhost:3001 | Express + Socket.io |
| Monad Testnet RPC | https://testnet-rpc.monad.xyz | Cek docs Monad untuk URL terbaru |
| Monad Explorer | https://testnet.monadexplorer.com | Cek tx dan kontrak |

## Dependencies yang Harus Diinstall

### Smart Contract

```
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox
npm install @openzeppelin/contracts  # untuk ownership, reentrancy guard
```

### Backend

```
npm install express socket.io ethers cors dotenv
npm install --save-dev nodemon
```

### Frontend

```
npx create-next-app@latest frontend --typescript --tailwind
npm install wagmi viem @tanstack/react-query
npm install socket.io-client
npm install @rainbow-me/rainbowkit  # atau ConnectKit untuk wallet modal
```

# Skenario

# Last Man Standing

## User Flow & Technical Specification

**Monad Blitz Jogja — 25 April 2026**

Dokumen Referensi Tim: SC · FE · BE

Versi 1.0

🔗 **Smart Contract** | 🖥️ **Frontend** | ⚙️ **Backend**

# BAGIAN 1 — User Flow Overview

Dokumen ini menggambarkan seluruh skenario penggunaan Last Man Standing dari sudut pandang setiap jenis user, beserta implikasi teknis di setiap langkahnya untuk Smart Contract (SC), Backend (BE), dan Frontend (FE).

## Jenis User dalam Sistem

| User Type | Deskripsi | Aksi Utama |
|---|---|---|
| 👤 Player | Pemain aktif yang join room dan bertarung | Join room, bayar fee, aktifkan shield, claim prize |
| 👁️ Spectator | Penonton yang tidak ikut bermain | Lihat arena, kirim reaksi, pasang prediksi |
| 🔧 Keeper / BE | Server backend otomatis | Trigger ronde, broadcast event, sinkronisasi state |
| 🏭 Room Creator | User yang membuat room baru (opsional) | Deploy room via factory, set konfigurasi |

# BAGIAN 2 — Skenario Player (Pemain)

## Skenario A — Player Menang 🏆

**1** **Buka dApp & Lihat Lobby**
User buka browser → akses URL dApp → melihat halaman lobby dengan daftar room aktif.

**2** **Connect Wallet**
Klik "Connect Wallet" → MetaMask popup → pilih akun → approve koneksi ke Monad testnet → saldo MON tampil di header.

**3** **Pilih & Lihat Room**
Lihat daftar room: status WAITING/ACTIVE/FINISHED, entry fee, jumlah pemain, prize pool. Klik Room #1 (WAITING).

**4** **Join Room & Bayar**
Klik "JOIN — 0.01 MON" → MetaMask konfirmasi tx → on-chain: joinRoom() dipanggil → player masuk game, shield aktif.

### Detail Teknis: joinRoom()

| Layer | Yang Terjadi |
|---|---|
| SC | msg.value diperiksa == entryFee. Player ditambahkan ke allPlayers[]. isAlive[player]=true, hasJoined[player]=true, hasShield[player]=true. prizePool += msg.value. |
| SC | Jika allPlayers.length >= maxPlayers → _startGame() otomatis dipanggil. |
| SC | emit PlayerJoined(address, totalPlayers, prizePool) |
| BE | Tangkap event PlayerJoined → broadcast socket "player_joined" ke semua client di room itu. |
| FE | UI update real-time: counter pemain naik, nama player baru muncul di grid, prize pool angkanya naik. |

**5** **Menunggu di Lobby**
Player lihat pemain lain masuk satu per satu. Prize pool naik. Setelah cukup pemain → keeper panggil startGame() → status ACTIVE → countdown dimulai.

### Detail Teknis: startGame()

| Layer | Yang Terjadi |
|---|---|
| SC | onlyKeeper: hanya backend keeper yang bisa panggil. status berubah WAITING → ACTIVE. gameStartTime dan lastRoundTime di-set ke block.timestamp. currentRound = 0. |
| SC | emit GameStarted(timestamp, totalPlayers, prizePool) |
| BE | Tangkap GameStarted → mulai countdown loop → setelah roundInterval detik, panggil executeRound() pertama kali. |
| FE | Tampilan berubah dari "lobby" ke "arena". Countdown timer muncul. Semua pemain tampil di grid berwarna hijau. |

**6**

**Ronde 1 — Eliminasi Pertama**

12 detik berlalu → keeper panggil executeRound() → 30% pemain dieliminasi secara acak → feed eliminasi muncul → player lihat dirinya masih hidup.

### Detail Teknis: executeRound() + _eliminatePlayers()

| Layer | Yang Terjadi |
|---|---|
| SC | currentRound++ . Cek block.timestamp >= lastRoundTime + roundInterval. |
| SC | _getRandomSeed(): keccak256(timestamp + prevrandao + roundNum + players.length + prizePool) → uint256 seed. |
| SC | _eliminatePlayers(): hitung elimCount = max(1, alive.length * 30% / 100). Fisher-Yates partial shuffle pakai seed. Untuk setiap yang terpilih eliminasi: cek shield. Jika hasShield && !shieldUsed → shieldUsed=true, emit ShieldBlocked. Jika tidak → isAlive=false, emit PlayerEliminated. |
| SC | Setelah eliminasi: cek getAlivePlayers().length == 1 → panggil _endGame(winner). |
| SC | emit RoundStarted(roundNum, aliveCount). emit PlayerEliminated x N. lastRoundTime = block.timestamp. |
| BE | Tangkap semua events → broadcast "round_started", "player_eliminated" ke semua client. |
| FE | Pemain gugur: warna di grid berubah merah lalu abu-abu. Feed: "💀 0x7f3... gugur di Ronde 1". Countdown ke ronde berikutnya reset. |

**7**

**Ronde 2 — Player Masih Hidup**

12 detik lagi → executeRound() lagi → 30% dari sisa dieliminasi → player masih selamat → semakin sedikit lawan.

**8**

**Ronde 3 — Shield Menyelamatkan Player**

Player terpilih dieliminasi, tapi kontrak deteksi hasShield=true, shieldUsed=false → shield dikonsumsi otomatis → player TIDAK jadi gugur → emit ShieldBlocked → FE tampilkan notifikasi dramatis.

### Detail Teknis: Shield Mechanic

| Layer | Yang Terjadi |
|---|---|
| SC | Di dalam loop _eliminatePlayers(), sebelum isAlive[addr]=false: if(hasShield[addr] && !shieldUsed[addr]) { shieldUsed[addr]=true; emit ShieldBlocked(addr,round); continue; } |
| SC | Shield tidak perlu diaktifkan manual. Otomatis dikonsumsi ketika terpilih eliminasi. |
| FE | Jika event ShieldBlocked dan address == currentUser → tampil overlay: "🛡️ SHIELD MENYELAMATKANMU!" dengan animasi dramatis. |
| FE | Badge shield di UI berubah dari "🛡️ Aktif" menjadi "🛡️ Terpakai" (abu-abu). |

**9**

**Ronde 4 — Tinggal 3 Pemain**

Eliminasi terus berjalan. Player sudah tidak punya shield. Tersisa 3 pemain. Tension meningkat.

**10**

**Ronde 5 — MENANG**

2 dari 3 pemain dieliminasi → player adalah 1 yang tersisa → kontrak panggil _endGame() → status FINISHED → emit GameFinished → WinnerScreen muncul.

### Detail Teknis: _endGame()

| Layer | Yang Terjadi |
|---|---|
| SC | winner = address yang tersisa. status = FINISHED. platformFee = prizePool * 5 / 100. Transfer platformFee ke treasury address. winnerPrize = prizePool - platformFee. claimed = false (siap untuk diklaim). |
| SC | emit GameFinished(winner, winnerPrize, currentRound) |
| BE | Tangkap GameFinished → stop keeper loop untuk room ini → broadcast "game_winner" ke semua client. |
| FE | WinnerScreen overlay fullscreen muncul dengan animasi konfeti. Tampil: nama pemenang, prize amount, total ronde. |

**11**

**Claim Hadiah**

Player klik "CLAIM PRIZE" → kontrak transfer winnerPrize langsung ke wallet → konfirmasi muncul.

### Detail Teknis: claimPrize()

| Layer | Yang Terjadi |
|---|---|
| SC | require(status == FINISHED). require(msg.sender == winner). require(!claimed). claimed = true. Transfer winnerPrize ke winner. |
| SC | emit PrizeClaimed(winner, amount) |
| FE | Setelah tx confirmed: notifikasi "✅ 0.114 MON sudah masuk ke wallet!" Balance wallet naik. |

## Skenario B — Player Gugur 💀

Langkah 1–7 identik dengan Skenario A. Perbedaan terjadi di momen eliminasi.

**8**

**Ronde 2 — Player Dieliminasi**

Shield sudah terpakai di ronde sebelumnya. Player terpilih di Fisher-Yates shuffle. isAlive[player]=false. Emit PlayerEliminated.

### Detail Teknis: Eliminasi Tanpa Shield

| Layer | Yang Terjadi |
|---|---|
| SC | Di _eliminatePlayers(): hasShield=true tapi shieldUsed=true → tidak ada perlindungan → isAlive[addr]=false → emit PlayerEliminated(addr, round, remaining). |
| BE | Broadcast "player_eliminated" dengan payload {player: addr, round: 2, remaining: 7}. |
| FE | Jika address == currentUser: overlay merah besar "💀 KAMU GUGUR DI RONDE 2". Setelah 3 detik otomatis switch ke Spectator Mode. |
| FE | Di Spectator Mode: tombol JOIN hilang, tombol shield hilang. Masih bisa lihat arena, kirim emoji, lihat siapa yang menang. |

**9**

**Mode Spectator Setelah Gugur**

Player yang gugur tetap bisa menonton. Lihat siapa yang bertahan. Kirim emoji reaksi. Tidak bisa claim prize.

📌 PENTING untuk FE:
Bedakan state "Pemain Aktif" vs "Spectator (ex-player)" vs "Spectator Murni".
Ex-player: sudah join (hasJoined=true), tapi isAlive=false.

Spectator murni: tidak pernah join room ini.
Keduanya tidak bisa klaim prize, tapi keduanya bisa kirim emoji dan lihat arena.

# BAGIAN 3 — Skenario Spectator (Penonton)

## Flow Spectator Murni

**1** **Buka dApp / Terima Share Link**
User buka URL dApp langsung, atau dapat link dari teman. Tidak perlu punya wallet (untuk lihat saja). Untuk prediksi perlu wallet.

**2** **Lihat Room dengan Status ACTIVE**
Di lobby, room yang sedang berlangsung ditandai "🔴 LIVE". Klik "SPECTATE" — tidak ada konfirmasi wallet, langsung masuk.

**3** **Lihat Arena**
Grid semua pemain: hijau=hidup, abu=gugur. Prize pool yang dipertarungkan. Ronde berapa. Countdown ke ronde berikutnya. Feed eliminasi.

**4** **Kirim Reaksi Emoji (Opsional)**
Klik emoji 🔥 😱 💀 👑 → BE broadcast ke semua spectator di room itu → emoji float di layar semua penonton.

### Detail Teknis: Spectator Emoji

| Layer | Yang Terjadi |
|---|---|
| BE | Terima socket event "spectator_react" dari FE: {emoji, from, roomAddress}. Broadcast kembali ke semua client di room itu. |
| FE | Animasi emoji float dari bawah layar ke atas selama 2 detik, lalu fade out. Tidak ada data yang disimpan on-chain. |
| SC | Tidak ada. Emoji adalah fitur off-chain murni. |

**5** **Pasang Prediksi (Opsional — jika aktif)**
Connect wallet → pilih pemain yang masih hidup → bet sejumlah MON → kontrak PredictionPool mencatat. Jika prediksi benar → dapat reward proporsional.

### Detail Teknis: Prediction Pool (Opsional)

| Layer | Yang Terjadi |
| --- | --- |
| SC | PredictionPool.sol (kontrak terpisah): placeBet(roomAddress, predictedWinner) payable. Simpan bet per address per room. Setelah GameFinished: distributeRewards() — bagi pool ke semua yang prediksi benar, proporsional dengan bet mereka. |
| BE | Listen GameFinished dari GameRoom → trigger distributeRewards() di PredictionPool. |
| FE | Panel prediksi: daftar pemain hidup + odds sederhana (berapa % orang bet di sini). Tombol bet + input amount. |

**6**

### Akhir Game — Lihat Pemenang

WinnerScreen muncul juga untuk spectator. Jika prediksi benar → notifikasi "🎉 Prediksi kamu benar! Klaim reward." Tombol "JOIN NEXT GAME".

# BAGIAN 4 — Skenario Keeper / Backend (Otomatis)

Keeper adalah komponen backend yang berjalan terus-menerus. Tugasnya: memastikan game berjalan otomatis tanpa intervensi manual.

## Lifecycle Backend

**1** **Server Startup**
Load ABI + contract addresses dari config/env. Connect ke Monad RPC dengan keeper wallet (punya MON untuk gas). Setup Socket.io server. Mulai listen events dari GameFactory.

**2** **Deteksi Room Baru**
Event RoomCreated dari factory → buat GameKeeper instance baru → mulai monitor room itu.

**3** **Monitor Room WAITING**
Listen PlayerJoined. Jika players.length >= minPlayers → panggil startGame() via keeper wallet.

**4** **Loop Ronde (ACTIVE)**
Setiap detik: cek apakah block.timestamp >= lastRoundTime + roundInterval. Jika ya: panggil executeRound(), tunggu tx confirmed, broadcast semua events.

**5** **Deteksi Game Selesai**
Tangkap event GameFinished → stop loop untuk room ini → broadcast "game_winner" ke FE.

**6** **Error Handling**
Tx gagal: retry 3x dengan exponential backoff. RPC down: reconnect otomatis. Game stuck > 2 jam: log alert, siap untuk emergency.

### Pseudocode Keeper Loop

```
async function keeperLoop(roomContract) {
  while (true) {
    const status = await roomContract.status();
    if (status === GameStatus.ACTIVE) {
      const lastRound = await roomContract.lastRoundTime();
      const interval = await roomContract.roundInterval();
      const now = Math.floor(Date.now() / 1000);
      if (now >= lastRound + interval) {
```

```
        try {
          const tx = await roomContract.executeRound();
          await tx.wait();
        } catch (err) {
          await sleep(3000); // retry setelah 3 detik
        }
      }
    }
    if (status === GameStatus.FINISHED) break; // stop loop
    await sleep(1000); // cek setiap 1 detik
  }
}
```

# BAGIAN 5 — Smart Contract: Apa yang Harus Dibuat

🔗 **Dikerjakan oleh: Lead Developer (SC)**

## File yang Harus Dibuat

| File | Status | Keterangan |
|---|---|---|
| contracts/GameRoom.sol | WAJIB | Kontrak utama: seluruh logika game |
| contracts/GameFactory.sol | WAJIB | Factory untuk deploy banyak room |
| contracts/IGameRoom.sol | WAJIB | Interface untuk interoperabilitas |
| contracts/ShieldSystem.sol | OPSIONAL | Bisa diintegrasikan langsung ke GameRoom |
| contracts/PredictionPool.sol | OPSIONAL | Kontrak terpisah untuk prediksi penonton |
| scripts/deploy.js | WAJIB | Script deploy ke Monad testnet |
| test/GameRoom.test.js | WAJIB | Unit test minimal |
| hardhat.config.js | WAJIB | Config dengan Monad network |

## GameRoom.sol — Semua State Variables

```
// ── CONFIGURATION (set saat constructor, tidak berubah) ──
uint256 public roomId;
uint256 public entryFee;          // dalam wei
uint256 public minPlayers;        // minimal pemain sebelum bisa start
uint256 public maxPlayers;        // batas maksimal pemain
uint256 public eliminationPct;    // % pemain yang dieliminasi per ronde (e.g. 30)
uint256 public roundInterval;     // detik antar ronde (e.g. 12)
uint256 public platformFeePct;    // % fee platform (e.g. 5)
address public treasury;          // penerima platform fee
address public keeper;            // address BE yang boleh panggil executeRound()
address public factory;           // factory yang deploy room ini

// ── GAME STATE ──
enum GameStatus { WAITING, COUNTDOWN, ACTIVE, FINISHED }
GameStatus public status;
address[] public allPlayers;
mapping(address => bool) public isAlive;
mapping(address => bool) public hasJoined;
address public winner;
uint256 public currentRound;
uint256 public lastRoundTime;
uint256 public gameStartTime;
uint256 public prizePool;
uint256 public winnerPrize;       // disimpan setelah _endGame()
```

```
bool    public prizeClaimed;

// ── SHIELD STATE ──
mapping(address => bool) public hasShield;
mapping(address => bool) public shieldUsed;
```

## GameRoom.sol — Semua Events

```
event PlayerJoined(address indexed player, uint256 totalPlayers, uint256
prizePool);
event GameStarted(uint256 timestamp, uint256 totalPlayers, uint256 prizePool);
event RoundStarted(uint256 roundNumber, uint256 playersAlive);
event PlayerEliminated(address indexed player, uint256 round, uint256
playersRemaining);
event ShieldBlocked(address indexed player, uint256 round);
event ShieldActivated(address indexed player, uint256 round); // opsional:
pre-signal
event GameFinished(address indexed winner, uint256 prize, uint256 totalRounds);
event PrizeClaimed(address indexed winner, uint256 amount);
event EmergencyWithdraw(address indexed by, uint256 amount);
```

## GameRoom.sol — Semua Functions

| Function | Visibility | Modifier | Deskripsi Singkat |
|---|---|---|---|
| constructor(...) | public | - | Set semua config, status = WAITING |
| joinRoom() | external payable | - | Join game, bayar fee, dapat shield, cek auto-start |
| startGame() | external | onlyKeeper | Ubah status ke ACTIVE, set timestamps |
| _startGame() | internal | - | Logic start game (dipanggil startGame & auto-start) |
| executeRound() | external | onlyKeeper | Jalankan 1 ronde eliminasi, update state |
| _getRandomSeed(roundNum) | internal view | - | Generate pseudo-random seed |
| _eliminatePlayers(seed, alive[]) | internal | - | Shuffle + eliminasi + cek shield |
| _endGame(winner) | internal | - | Set pemenang, hitung fee, ubah status FINISHED |
| claimPrize() | external | nonReentrant | Winner klaim hadiah |
| activateShield() | external | - | Signal pre-aktivasi shield (opsional) |

| Function | Visibility | Modifier | Deskripsi Singkat |
|---|---|---|---|
| emergencyWithdraw() | external | onlyOwner | Kembalikan dana jika game stuck |
| getAlivePlayers() | public view | - | Return array address yang masih hidup |
| getAllPlayers() | external view | - | Return semua address yang pernah join |
| getPlayerCount() | external view | - | Return (total, alive) count |
| getGameInfo() | external view | - | Return semua state penting dalam 1 call |
| isPlayerAlive(addr) | external view | - | Cek apakah address masih hidup |
| getShieldStatus(addr) | external view | - | Return (hasShield, shieldUsed) |

## GameRoom.sol — Logika Kritis yang Harus Benar

### 1. Logika joinRoom()

```
function joinRoom() external payable {
    require(status == GameStatus.WAITING, "Game sudah dimulai");
    require(msg.value == entryFee, "Entry fee tidak tepat");
    require(!hasJoined[msg.sender], "Sudah join");
    require(allPlayers.length < maxPlayers, "Room penuh");

    allPlayers.push(msg.sender);
    isAlive[msg.sender] = true;
    hasJoined[msg.sender] = true;
    hasShield[msg.sender] = true;    // shield gratis saat join
    prizePool += msg.value;

    emit PlayerJoined(msg.sender, allPlayers.length, prizePool);

    // Auto-start jika sudah penuh
    if (allPlayers.length >= maxPlayers) {
        _startGame();
    }
}
```

### 2. Logika _eliminatePlayers() — Fisher-Yates + Shield Check

```
function _eliminatePlayers(uint256 seed) internal {
    address[] memory alive = getAlivePlayers();
    uint256 elimCount = (alive.length * eliminationPct) / 100;
```

```
    if (elimCount == 0) elimCount = 1;
    // KRITIS: jangan sampai eliminasi semua → harus ada 1 tersisa
    if (alive.length - elimCount == 0) elimCount = alive.length - 1;

    // Fisher-Yates partial shuffle
    for (uint256 i = 0; i < elimCount; i++) {
        uint256 rand = uint256(keccak256(abi.encodePacked(seed, i)));
        uint256 j = i + (rand % (alive.length - i));
        // swap
        address tmp = alive[i]; alive[i] = alive[j]; alive[j] = tmp;

        address target = alive[i];

        // Cek shield
        if (hasShield[target] && !shieldUsed[target]) {
            shieldUsed[target] = true;
            emit ShieldBlocked(target, currentRound);
            // JANGAN set isAlive=false → player selamat
        } else {
            isAlive[target] = false;
            uint256 remaining = getAlivePlayers().length;
            emit PlayerEliminated(target, currentRound, remaining);
        }
    }

    // Cek kondisi menang
    address[] memory survivors = getAlivePlayers();
    if (survivors.length == 1) {
        _endGame(survivors[0]);
    }
}
```

### 3. Logika _endGame()

```
function _endGame(address _winner) internal {
    winner = _winner;
    status = GameStatus.FINISHED;

    uint256 platformFee = (prizePool * platformFeePct) / 100;
    winnerPrize = prizePool - platformFee;

    // Transfer fee ke treasury
    if (platformFee > 0) {
        (bool sent,) = treasury.call{value: platformFee}("");
        require(sent, "Fee transfer gagal");
    }

    emit GameFinished(_winner, winnerPrize, currentRound);
}
```

## GameFactory.sol — Spesifikasi

| Function | Deskripsi |
|---|---|
| constructor(treasury, keeper) | Set default treasury dan keeper address |
| createRoom(entryFee, minP, maxP, elimPct, interval) | Deploy GameRoom baru, push ke allRooms[], emit RoomCreated |
| getActiveRooms() | Filter dan return room dengan status WAITING atau ACTIVE |
| getAllRooms() | Return semua room address |
| setKeeper(newKeeper) | onlyOwner: ganti keeper address |
| setTreasury(newTreasury) | onlyOwner: ganti treasury address |

## Output SC yang Harus Diserahkan ke Tim

| Output | Format | Untuk | Kapan |
|---|---|---|---|
| GameFactory address | String 0x... | FE + BE | Setelah deploy |
| GameRoom ABI | gameRoom.abi.json | FE + BE | Setelah compile |
| GameFactory ABI | gameFactory.abi.jso n | FE + BE | Setelah compile |
| Semua event signatures | Tabel / README | BE (listener) | Hari 1 pagi |
| Test wallet addresses | List address + private key | FE demo | Hari 2 |
| Monad RPC + Chain ID | String URL + number | FE + BE | Segera |

# BAGIAN 6 — Frontend: Apa yang Harus Dibuat

🖥️ **Dikerjakan oleh: Frontend Developer**

## Halaman yang Harus Dibuat

| Halaman | Route | Status | Deskripsi |
|---|---|---|---|
| Landing / Lobby | / (index) | WAJIB | List semua room aktif, connect wallet |
| Arena / Room | /room/[address] | WAJIB | Halaman utama game: grid pemain, feed, timer |
| Winner Screen | (overlay di Arena) | WAJIB | Muncul fullscreen saat game selesai |
| Spectator View | (state di Arena) | WAJIB | Mode view-only untuk non-player atau yang gugur |
| Create Room | /create (opsional) | OPSIONAL | Form untuk deploy room baru via factory |

## Komponen yang Harus Dibuat

| Komponen | Status | Dipanggil di | Fungsi |
|---|---|---|---|
| WalletConnect.tsx | WAJIB | Semua halaman | Tombol connect + status wallet |
| RoomCard.tsx | WAJIB | Lobby | Card info room: fee, pemain, status, prize |
| PlayerGrid.tsx | WAJIB | Arena | Grid semua pemain + warna status (hidup/gugur) |
| CountdownTimer.tsx | WAJIB | Arena | Hitung mundur ke ronde berikutnya |
| EliminationFeed.tsx | WAJIB | Arena | Feed real-time event eliminasi |
| StatusBadge.tsx | WAJIB | Arena | Badge status player: HIDUP / GUGUR / MENANG |
| ShieldIndicator.tsx | WAJIB | Arena | Indikator shield: aktif / terpakai |
| WinnerScreen.tsx | WAJIB | Arena (overlay) | Screen pemenang + tombol claim |
| JoinButton.tsx | WAJIB | Arena | Tombol join + trigger tx on-chain |
| SpectatorPanel.tsx | OPSIONAL | Arena | Panel prediksi + emoji reaksi |
| EmojiReaction.tsx | OPSIONAL | Arena | Tombol + animasi float emoji |

## Hooks yang Harus Dibuat

| Hook | Status | Fungsi |
|---|---|---|
| useWallet.ts | WAJIB | State wallet: address, balance, isConnected, chainId |
| useGameRoom.ts | WAJIB | Read contract: alivePlayers, gameStatus, prizePool (watch=true) |
| useSocket.ts | WAJIB | Subscribe Socket.io events: eliminated, round_started, winner |
| useJoinRoom.ts | WAJIB | Write contract: joinRoom() dengan value entryFee |
| useClaimPrize.ts | WAJIB | Write contract: claimPrize() |
| useShield.ts | OPSIONAL | Read: getShieldStatus(). Write: activateShield() |
| usePrediction.ts | OPSIONAL | Write: placeBet(). Read: getOdds() |

## State Management — Apa yang Harus Di-track di FE

```
// State global yang perlu dikelola (bisa pakai Zustand / Context / useState)

interface GameState {
  // Wallet
  walletAddress: string | null;
  isConnected: boolean;

  // Room
  roomAddress: string;
  gameStatus: "WAITING" | "ACTIVE" | "FINISHED";
  currentRound: number;
  prizePool: string;    // dalam ETH/MON string

  // Players
  allPlayers: string[];
  alivePlayers: string[];
  eliminatedPlayers: string[];

  // Current user state
  isCurrentUserAlive: boolean;
  hasCurrentUserJoined: boolean;
  currentUserHasShield: boolean;
  currentUserShieldUsed: boolean;

  // Feed
  feedEvents: FeedEvent[];  // [{type, player, round, timestamp}]

  // Winner
  winner: string | null;
  winnerPrize: string | null;
  hasClaimed: boolean;
}
```

## Integrasi Socket.io di FE — Yang Harus Didengarkan

| Event Nama | Payload | Update State FE |
|---|---|---|
| player_joined | {player, totalPlayers, prizePool} | Tambah ke allPlayers[], update prize pool |
| game_started | {timestamp, totalPlayers, prizePool} | Set status=ACTIVE, munculkan countdown |
| round_started | {round, playersAlive} | Update currentRound, reset countdown timer |
| player_eliminated | {player, round, remaining} | Pindah dari alive ke eliminated, animasi grid |
| shield_blocked | {player, round} | Jika currentUser: tampil notifikasi shield. Update shieldUsed=true |
| game_winner | {winner, prize} | Set winner, tampilkan WinnerScreen overlay |
| spectator_react | {emoji, from} | Tampilkan animasi emoji float di arena |
| game_status | {status, round, alive} | Sync state berkala (fallback jika miss event) |

## Integrasi Kontrak di FE — Yang Harus Di-call

| Aksi User | Function SC | Tipe | Value |
|---|---|---|---|
| Join room | joinRoom() | write payable | entryFee (dari getGameInfo) |
| Claim prize | claimPrize() | write | 0 (tidak bayar) |
| Aktifkan shield | activateShield() | write | 0 (opsional) |
| Lihat pemain hidup | getAlivePlayers() | read | - |
| Lihat info game | getGameInfo() | read | - |
| Cek status shield | getShieldStatus(addr) | read | - |
| List room | GameFactory.getActiveRooms() | read | - |

## UX Checklist — Kondisi yang Harus Di-handle

| Kondisi | Yang Ditampilkan |
|---|---|
| Wallet belum connect | Prompt connect wallet, semua tombol disabled |

| Kondisi | Yang Ditampilkan |
|---|---|
| Chain ID salah (bukan Monad) | Banner "Switch ke Monad Testnet" + tombol auto-switch |
| Status WAITING | Lobby: list pemain, prize pool, tombol JOIN, countdown ke auto-start |
| Status ACTIVE + user hidup | Arena: grid pemain, feed, countdown ronde, badge "🟢 HIDUP", shield indicator |
| Status ACTIVE + user gugur | Arena sama + overlay merah "💀 GUGUR", semua tombol aksi disabled |
| Status ACTIVE + spectator murni | Arena view-only, tidak ada tombol JOIN (sudah closed), ada panel reaksi |
| Status FINISHED + user menang | WinnerScreen fullscreen + tombol CLAIM PRIZE |
| Status FINISHED + user kalah | Layar simpel "Kamu tidak menang" + tombol join game baru |
| Tx pending | Loading state di tombol, disable double-click |
| Tx gagal | Toast error dengan pesan yang jelas |

# BAGIAN 7 — Backend: Apa yang Harus Dibuat

⚙️ **Dikerjakan oleh: Backend Developer**

## File yang Harus Dibuat

| File | Status | Fungsi |
|---|---|---|
| src/index.js | WAJIB | Entry point: init server, socket, keeper |
| src/config.js | WAJIB | Load env: RPC URL, contract address, ABI, port |
| src/keeper/GameKeeper.js | WAJIB | Loop utama: trigger executeRound() tiap interval |
| src/keeper/KeeperManager.js | WAJIB | Manage banyak GameKeeper instance (1 per room) |
| src/chain/ContractListener.js | WAJIB | Listen semua events on-chain → broadcast socket |
| src/chain/provider.js | WAJIB | Setup ethers provider + signer (keeper wallet) |
| src/socket/socketHandler.js | WAJIB | Handle koneksi Socket.io client, room management |
| src/routes/rooms.js | WAJIB | REST API: GET /rooms, GET /rooms/:address |
| src/routes/admin.js | OPSIONAL | Trigger manual startGame, emergencyWithdraw |
| shared/gameRoom.abi.json | WAJIB | ABI dari SC (di-copy dari artifacts/) |
| shared/gameFactory.abi.json | WAJIB | ABI factory dari SC |

## Semua Socket.io Events — Kontrak dengan FE

### Events yang Di-broadcast BE → FE

| Event Name | Kapan | Payload Lengkap |
|---|---|---|
| player_joined | Setiap PlayerJoined event on-chain | {roomAddress, player, totalPlayers, prizePool, timestamp} |
| game_started | Setiap GameStarted event on-chain | {roomAddress, timestamp, totalPlayers, prizePool} |
| round_started | Setiap RoundStarted event on-chain | {roomAddress, round, playersAlive, nextRoundAt} |
| player_eliminated | Setiap PlayerEliminated event on-chain | {roomAddress, player, round, playersRemaining} |
| shield_blocked | Setiap ShieldBlocked event on-chain | {roomAddress, player, round} |

| Event Name | Kapan | Payload Lengkap |
|---|---|---|
| game_winner | Setiap GameFinished event on-chain | {roomAddress, winner, prize, totalRounds} |
| game_status | Polling setiap 5 detik (fallback) | {roomAddress, status, round, alive, prizePool} |

### Events yang Diterima BE dari FE

| Event Name | Dari | Aksi BE |
|---|---|---|
| spectator_react | {emoji, from, roomAddress} | Broadcast kembali ke semua client di room itu |
| join_room_socket | {roomAddress, address} | Add client ke Socket room channel (bukan join SC) |
| leave_room_socket | {roomAddress} | Remove client dari Socket room channel |

## REST API Endpoints

| Method | Endpoint | Response | Keterangan |
|---|---|---|---|
| GET | /api/rooms | [{address, status, players, prizePool, entryFee}] | List semua room dari factory |
| GET | /api/rooms/:addr | {status, currentRound, prizePool, alivePlayers[], allPlayers[]} | Detail satu room |
| GET | /api/rooms/:addr/players | [{address, isAlive, hasShield, shieldUsed}] | Status semua pemain |
| POST | /api/admin/start/:addr | {success, txHash} | Manual trigger startGame (admin only) |

## ContractListener.js — Setup Event Listening

```
function setupListeners(contract, io, roomAddress) {

  contract.on("PlayerJoined", (player, total, pool, event) => {
    io.to(roomAddress).emit("player_joined", {
      roomAddress, player,
      totalPlayers: total.toString(),
      prizePool: ethers.formatEther(pool),
      timestamp: Date.now()
    });
  });

  contract.on("PlayerEliminated", (player, round, remaining, event) => {
```

```
    io.to(roomAddress).emit("player_eliminated", {
      roomAddress, player,
      round: round.toString(),
      playersRemaining: remaining.toString(),
      timestamp: Date.now()
    });
  });

  contract.on("ShieldBlocked", (player, round, event) => {
    io.to(roomAddress).emit("shield_blocked", { roomAddress, player, round:
round.toString() });
  });

  contract.on("GameFinished", (winner, prize, rounds, event) => {
    io.to(roomAddress).emit("game_winner", {
      roomAddress, winner,
      prize: ethers.formatEther(prize),
      totalRounds: rounds.toString()
    });
    keeperManager.stopKeeper(roomAddress); // stop loop
  });
}
```

## GameKeeper.js — Core Keeper Class

```
class GameKeeper {
  constructor(roomAddress, contract, io) {
    this.roomAddress = roomAddress;
    this.contract = contract;
    this.io = io;
    this.loopHandle = null;
    this.isRunning = false;
  }

  start() {
    this.isRunning = true;
    this._loop();
  }

  async _loop() {
    while (this.isRunning) {
      try {
        const status = await this.contract.status();

        if (status === 0n) {  // WAITING
          const count = await this.contract.allPlayers.length;
          const min = await this.contract.minPlayers();
          if (count >= min) {
            await (await this.contract.startGame()).wait();
          }
```

```
        }

        if (status === 2n) {  // ACTIVE
          const lastRound = await this.contract.lastRoundTime();
          const interval = await this.contract.roundInterval();
          const now = BigInt(Math.floor(Date.now() / 1000));
          if (now >= lastRound + interval) {
            await (await this.contract.executeRound()).wait();
          }
        }

        if (status === 3n) {  // FINISHED
          this.stop();
          break;
        }
      } catch (err) {
        console.error("Keeper error:", err.message);
      }
      await this._sleep(1000);
    }
  }

  stop() { this.isRunning = false; }
  _sleep(ms) { return new Promise(r => setTimeout(r, ms)); }
}
```

## Environment Variables yang Dibutuhkan BE

```
# .env (Backend)
PORT=3001
MONAD_RPC_URL=https://testnet-rpc.monad.xyz
KEEPER_PRIVATE_KEY=0x...    # wallet yang punya MON untuk gas
GAME_FACTORY_ADDRESS=0x...  # di-isi setelah SC deploy
PLATFORM_FEE_PCT=5
CORS_ORIGIN=http://localhost:3000
```

# BAGIAN 8 — Peta Integrasi Antar Layer

## Diagram Alur Data (Teks)

```
┌──────────────────────────────────────┐
│          MONAD BLOCKCHAIN            │
│                                      │
│  GameFactory.sol                     │
│  ├── createRoom() → GameRoom.sol     │
│  └── emit RoomCreated                │
│                                      │
│  GameRoom.sol                        │
│  ├── joinRoom() ← [FE user tx]       │
│  ├── startGame() ← [BE keeper]       │
│  ├── executeRound() ← [BE keeper]    │
│  ├── claimPrize() ← [FE user tx]     │
│  └── emit events → [BE listener]     │
└──────────────────────────────────────┘
                │
                │ events
                ▼
┌──────────────────────────────────────┐
│            BACKEND (Node.js)         │
│                                      │
│  ContractListener → listen events    │
│  GameKeeper → trigger executeRound   │
│  Socket.io → broadcast ke FE         │
│  REST API → serve room data          │
└──────────────────────────────────────┘
                │
                │ socket events + REST
                ▼
┌──────────────────────────────────────┐
│            FRONTEND (Next.js)        │
│                                      │
│  useSocket → subscribe events        │
│  useGameRoom → read contract state   │
│  wagmi → send user transactions      │
│  UI → render game state              │
└──────────────────────────────────────┘
```

## Tabel Dependency Antar Tim

| Tim | Butuh Dari SC | Butuh Dari BE | Yang Diberikan ke Tim Lain |
|---|---|---|---|
| SC (Lead) | — | — | ABI files, contract addresses, event list, test wallets |
| Backend | ABI + address untuk listener & keeper | — | Socket event names + |

| Tim | Butuh Dari SC | Butuh Dari BE | Yang Diberikan ke Tim Lain |
|---|---|---|---|
| | | | payload, REST API docs, server URL |
| Frontend | ABI + address untuk write tx | Socket events + REST API | Demo UI, user testing |

## Urutan Handoff yang Wajib Diikuti

| # | Aksi | Dari | Ke | Target Waktu |
|---|---|---|---|---|
| 1 | Share event signatures list + ABI dummy (sebelum deploy) | SC | BE | Hari 1, 09:00 |
| 2 | BE setup listener + socket skeleton (pakai ABI dummy dulu) | BE | — | Hari 1, 10:00 |
| 3 | FE setup wallet connect + socket subscribe (pakai mock data) | FE | — | Hari 1, 10:00 |
| 4 | SC deploy → share address + ABI final | SC | FE + BE | Hari 1, 13:00 |
| 5 | BE ganti ABI dummy → ABI final, restart, konfirmasi listener jalan | BE | — | Hari 1, 13:30 |
| 6 | FE ganti ABI dummy → ABI final, test joinRoom() on-chain | FE | — | Hari 1, 14:00 |
| 7 | End-to-end test: full game loop dengan 3 wallet | Semua | — | Hari 1, 17:00 |

## Shared .env Template (Semua Tim)

```
# ─── SHARED CONFIG (bagikan ke seluruh tim) ──────────────────
MONAD_RPC_URL=https://testnet-rpc.monad.xyz
CHAIN_ID=10143

# Diisi SC setelah deploy
GAME_FACTORY_ADDRESS=0x_FILL_AFTER_DEPLOY

# Backend server
BACKEND_URL=http://localhost:3001
```

```
# ─── SC ONLY (jangan share ke FE/BE) ──────────────────────
PRIVATE_KEY=0x_SC_DEPLOYER_KEY

# ─── BE ONLY ──────────────────────────────────────────────
KEEPER_PRIVATE_KEY=0x_KEEPER_KEY
PORT=3001
CORS_ORIGIN=http://localhost:3000

# ─── FE ONLY ──────────────────────────────────────────────
NEXT_PUBLIC_BACKEND_URL=http://localhost:3001
NEXT_PUBLIC_CHAIN_ID=10143
NEXT_PUBLIC_FACTORY_ADDRESS=0x_FILL_AFTER_DEPLOY
```

## Checklist End-to-End Test (Hari 1 Sore)

| # | Test Case | Dicek Oleh | Pass? |
|---|---|---|---|
| 1 | Wallet connect ke Monad testnet berhasil | FE | [ ] |
| 2 | GET /api/rooms return list room | BE + FE | [ ] |
| 3 | joinRoom() sukses, tx confirmed on-chain | FE + SC | [ ] |
| 4 | PlayerJoined event di-broadcast via socket | BE + FE | [ ] |
| 5 | startGame() dipanggil keeper setelah cukup player | BE + SC | [ ] |
| 6 | executeRound() dijalankan tiap ~12 detik | BE + SC | [ ] |
| 7 | PlayerEliminated event muncul di feed FE | BE + FE | [ ] |
| 8 | Shield memblokir eliminasi (ShieldBlocked event muncul) | SC + FE | [ ] |
| 9 | WinnerScreen muncul otomatis saat game selesai | FE | [ ] |
| 10 | claimPrize() sukses, balance wallet pemenang naik | FE + SC | [ ] |

# BAGIAN 9 — Referensi Cepat

## Semua Events SC yang Harus Di-handle

| Event | Parameter | BE Listen? | FE React? | Penting? |
|---|---|---|---|---|
| PlayerJoined | player, totalPlayers, prizePool | ✅ | Update grid | WAJIB |
| GameStarted | timestamp, totalPlayers, prizePool | ✅ | Tampil arena | WAJIB |
| RoundStarted | roundNumber, playersAlive | ✅ | Reset timer | WAJIB |
| PlayerEliminated | player, round, remaining | ✅ | Animasi gugur | WAJIB |
| ShieldBlocked | player, round | ✅ | Notif shield | WAJIB |
| GameFinished | winner, prize, rounds | ✅ | WinnerScreen | WAJIB |
| PrizeClaimed | winner, amount | ✅ | Konfirmasi | WAJIB |
| EmergencyWithdraw | by, amount | ✅ | Alert admin | OPSIONAL |

## Ringkasan Fitur — Wajib vs Opsional

| Fitur | SC | BE | FE | Status |
|---|---|---|---|---|
| Join room + bayar fee | joinRoom() | listen PlayerJoined | JoinButton.tsx | WAJIB |
| Start game otomatis | startGame() | keeper trigger | - | WAJIB |
| Eliminasi per ronde | executeRound() | keeper loop | EliminationFeed.tsx | WAJIB |
| Shield block eliminasi | hasShield logic | listen ShieldBlocked | ShieldIndicator.tsx | WAJIB |
| Claim prize | claimPrize() | listen PrizeClaimed | WinnerScreen.tsx | WAJIB |
| Spectator mode | - | socket rooms | Arena view-only | WAJIB |
| Emoji reaksi | - | broadcast react | EmojiReaction.tsx | OPSIONAL |
| Prediksi winner | PredictionPool.sol | distribusi reward | SpectatorPanel.tsx | OPSIONAL |
| Create room dari UI | GameFactory | listen RoomCreated | CreateRoom page | OPSIONAL |

# Info Monad

**Link**
https://luma.com/blitz-jogja-apr-2026

**judgment**

**Voting Weightage:** The final evaluation will be based on a **50-50 split — 50% votes from participants** and **50% from jury members** to ensure balanced and fair judgment.

**Persiapan**

**Navigating Monad Blitz**

- Rules & Guidelines [IMPORTANT, PLEASE READ]
- What to Bring
- Preparing for Your Project Demo
- Blitz Guide - How to 360 no scope one-shot your idea into a reality?
- Resources
- Submission Process
- Judging Process & Criteria

**Syarat projek & ide**

- **Fresh Ideas Only (No Pre-Built Projects):**
  - All projects must be new and conceived specifically for Monad Blitz.
  - You **cannot** submit existing projects, fork existing codebases (beyond standard libraries/boilerplates), or continue work on personal projects you've already started. The spirit is to build something fresh.

**Fokus projek**
1. Games (e.g. randomness, gambling)
2. Social Fi
3. x402
4. Prediction market
5. Consumer focus/user/market acquisition

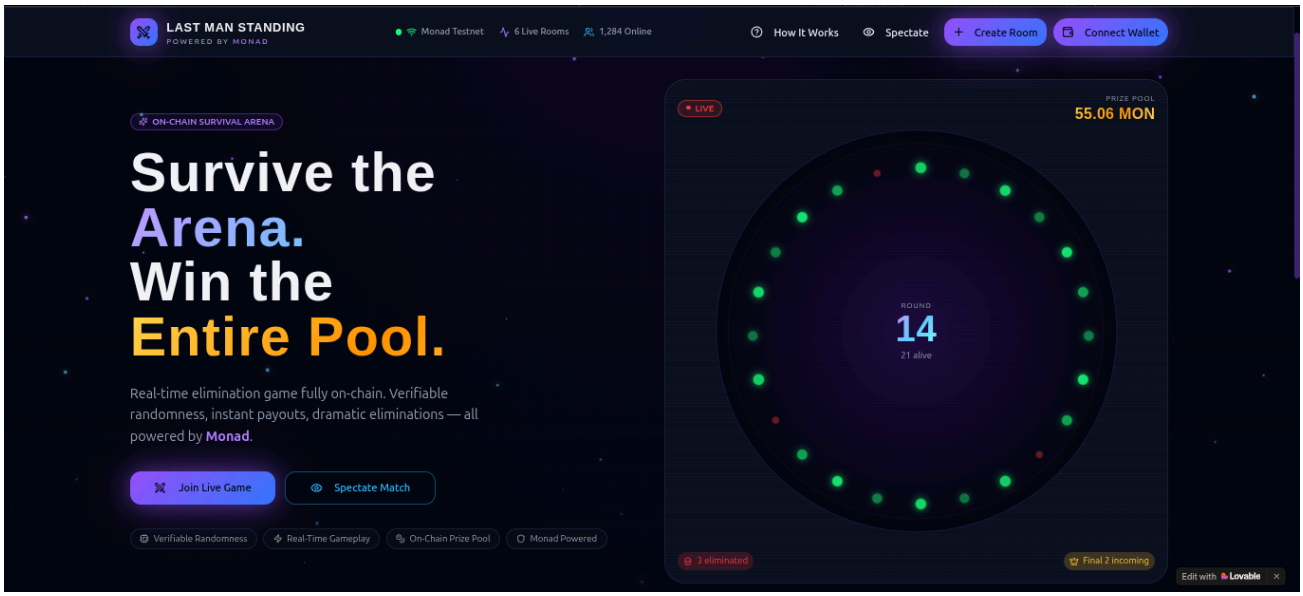
**cara ngumpulin**
https://monad-foundation.notion.site/Submission-Process-3486367594f28169be85dfed86ce08dc

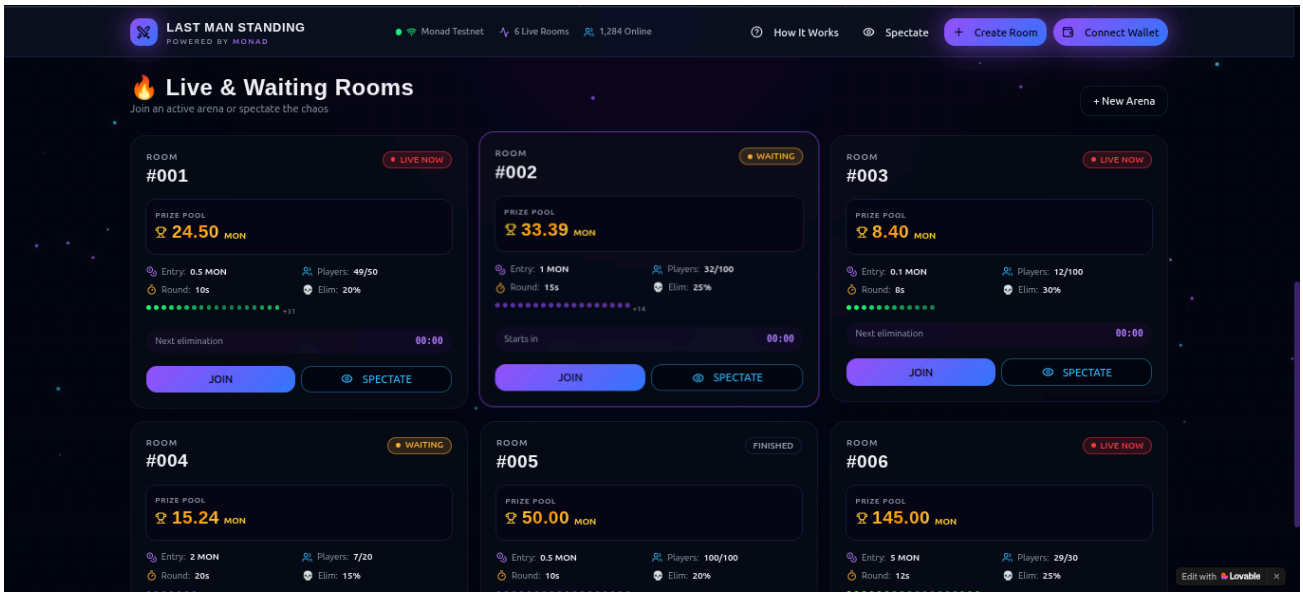
perhatikan cara submit pas monad besok ya teman teman, harus PR dan submit ke blitz

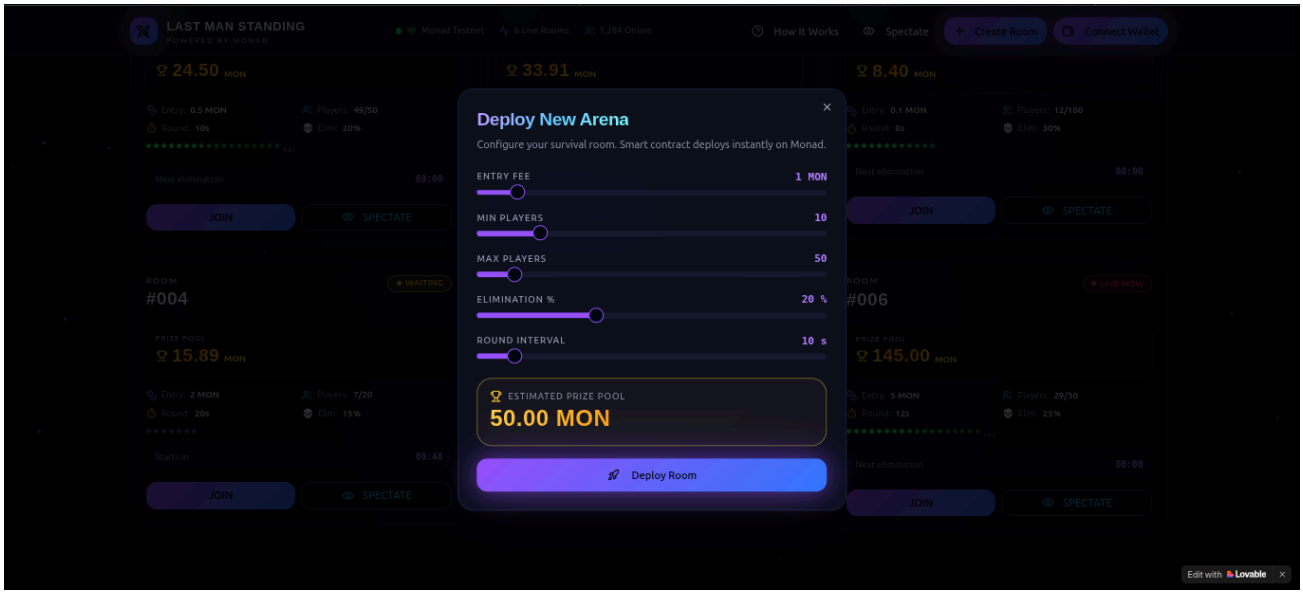
prototype

https://last-stand-monad.lovable.app/

github: c

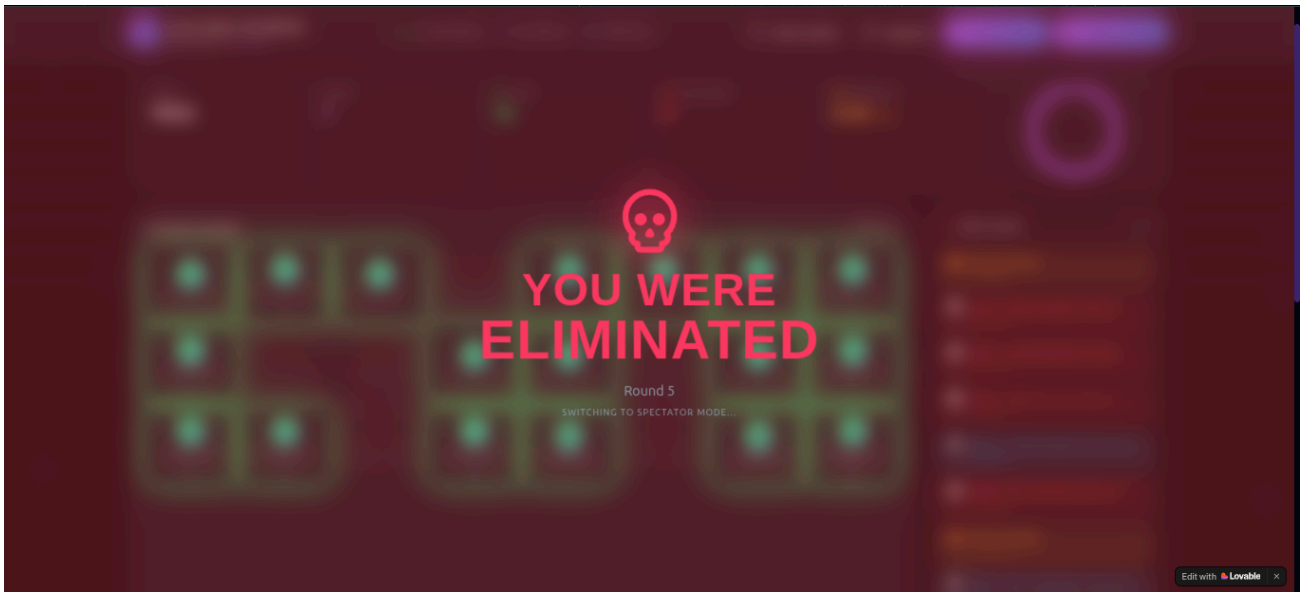


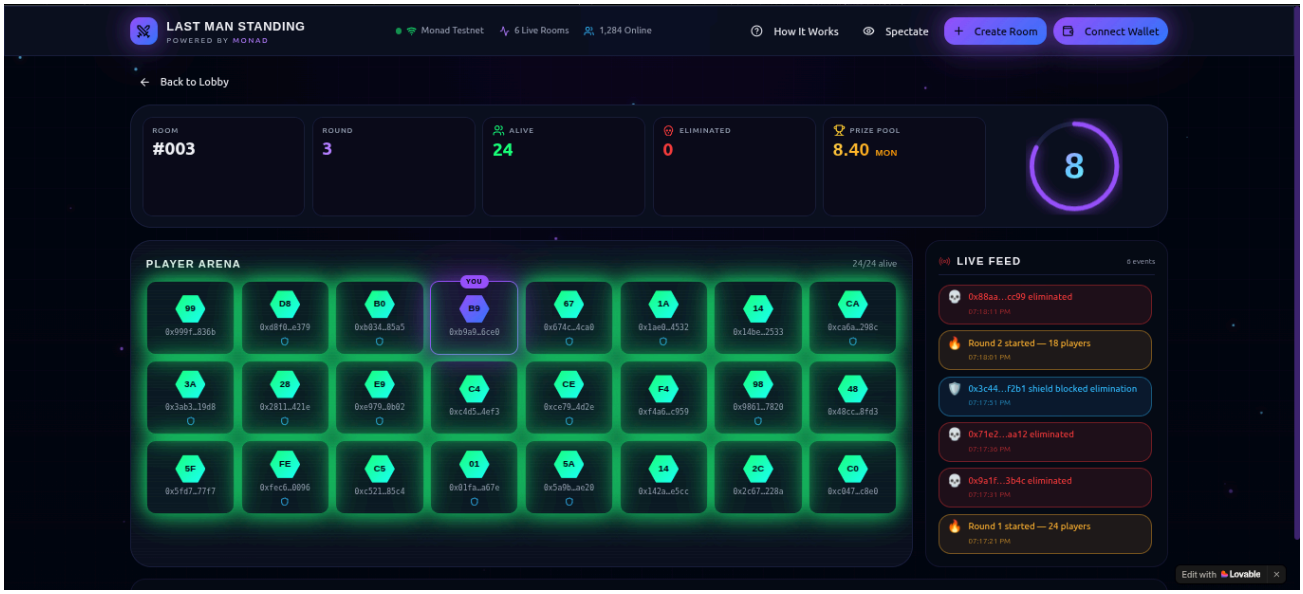




LAST MAN STANDING
POWERED BY MONAD
Monad Testnet
6 Live Rooms
1,284 Online
How It Works
Spectate
Create Room
Connect Wallet
24.50 MON
33.91 MON
8.40 MON
Entry: 0.5 MON
Round: 10s
Players: 49/50
Elim: 20%
Next elimination
00:00
JOIN
SPECTATE
ROOM
#004
WAITING
PRIZE POOL
15.89 MON
Entry: 2 MON
Round: 20s
Players: 7/20
Elim: 15%
Starts in
00:48
JOIN
SPECTATE
Deploy New Arena
Configure your survival room. Smart contract deploys instantly on Monad.
ENTRY FEE
1 MON
MIN PLAYERS
10
MAX PLAYERS
50
ELIMINATION %
20 %
ROUND INTERVAL
10 s
ESTIMATED PRIZE POOL
50.00 MON
Deploy Room
Entry: 0.1 MON
Round: 8s
Players: 12/100
Elim: 30%
Next elimination
00:00
JOIN
SPECTATE
ROOM
#006
LIVE NOW
PRIZE POOL
145.00 MON
Entry: 5 MON
Round: 12s
Players: 29/30
Elim: 25%
Next elimination
00:00
JOIN
SPECTATE
Edit with Lovable

MATCH COMPLETE
WINNER
0x1999...4eda
ROUNDS SURVIVED
20
PRIZE
28.34 MON
Claim Prize
Play Again
Edit with Lovable

LAST MAN STANDING
POWERED BY MONAD
Monad Testnet
6 Live Rooms
1,284 Online
How It Works
Spectate
Create Room
Connect Wallet
Back to Lobby
ROOM
#001
ROUND
4
ALIVE
21
ELIMINATED
3
PRIZE POOL
24.75 MON
9
PLAYER ARENA
21/24 alive
YOU
LIVE FEED
13 events
Round 5 starting...
SHIELD SAVED YOU
Edit with Lovable


YOU WERE
ELIMINATED
Round 5
SWITCHING TO SPECTATOR MODE...
Edit with Lovable


LAST MAN STANDING
POWERED BY MONAD
Monad Testnet
6 Live Rooms
1,284 Online
How It Works
Spectate
Create Room
Connect Wallet
Back to Lobby
ROOM
#003
ROUND
3
ALIVE
24
ELIMINATED
0
PRIZE POOL
8.40 MON
8
PLAYER ARENA
24/24 alive
YOU
LIVE FEED
6 events
0x88aa...cc99 eliminated
07:18:11 PM
Round 2 started — 18 players
07:18:01 PM
0x3c44...f2b1 shield blocked elimination
07:17:31 PM
0x71e2...aa12 eliminated
07:17:36 PM
0x9a1f...3b4c eliminated
07:17:31 PM
Round 1 started — 24 players
07:17:21 PM
Edit with Lovable

# Inspirasi desain web

# 🎮 LAST MAN STANDING — UI Reference Masterlist

### Curated Visual Inspiration untuk Hackathon-Winning Frontend

Berdasarkan gambar project kamu (lobby room grid + arena player hexagon + live feed), berikut adalah **30+ referensi terbaik** yang dikurasi khusus untuk membangun frontend "Last Man Standing" di Monad:

---

## 🔥 TIER S — REFERENSI PALING RELEVAN (Mirip Langsung dengan Projek)

---

### 1. POLYMARKET

**Link:** https://polymarket.com **GitHub:** https://github.com/Polymarket (internal/private) **Type:** Prediction Market UI **Why It's Relevant:** Mirip persis kebutuhan "spectator predict winner" — card-based layout untuk active markets, live indicator (`● LIVE`), probability bar Yes/No, real-time price update. Room card #001/#002/#003-mu sangat terinspirasi dari layout card Polymarket. **Best Elements:** Live badge, dark card UI, percentage probability bars, market status filter (All/Trending), volume info per card, real-time updating values **Visual Style:** Minimalist Dark + Premium Web3 **Reusability Score:** 10/10

---

### 2. DEXSCREENER

**Link:** https://dexscreener.com **GitHub:** https://github.com/dexscreener (private) **Type:** Realtime Crypto Dashboard **Why It's Relevant:** Realtime data feed dengan angka yang terus berubah (seperti prize pool MON yang naik), warna merah/hijau untuk perubahan, ticker yang scrolling, tab filter "Trending / Live / New" — persis seperti lobby room filtermu **Best Elements:** Live ticker bar di atas, tabel realtime yang update otomatis, color-coded change indicators, dark high-contrast theme, neon highlight pada angka penting **Visual Style:** Dark + Data-Intensive + Realtime **Reusability Score:** 9/10

---

### 3. PUMP.FUN

**Link:** https://pump.fun **GitHub:** N/A (proprietary) **Type:** Web3 GameFi / Realtime Token Launch **Why It's Relevant:** Inilah referensi **paling identik** dengan vibes projek kamu. Pump.fun punya: Live tab dengan coin cards yang terus muncul baru (seperti room baru dibuka), status badge "LIVE

/ WAITING / FINISHED", tombol aksi (Join / Spectate), volume & player count per card, tab "Mayhem / Live / New" persis seperti room filter kamu **Best Elements:** Card grid realtime, "Live" filter tab, status badge berwarna, quick-join UX, sidebar chat/live feed, animated card appearance **Visual Style:** Dark + Crypto-Native + Chaotic Energy **Reusability Score:** 10/10

---

### 4. HYPERLIQUID (App)

**Link:** https://app.hyperliquid.xyz **GitHub:** N/A **Type:** On-chain Trading / Realtime Dashboard **Why It's Relevant:** Hyperliquid adalah benchmark terbaik untuk "premium Web3 realtime UI" — semua data update per millisecond, countdown timer pada order, leaderboard trader (bisa diadaptasi jadi player leaderboard), live liquidation feed (mirip live elimination feed-mu) **Best Elements:** Realtime number animation, liquidation feed (inspiration untuk elimination feed), leaderboard panel, countdown timer, glowing neon accent pada angka, wallet-connected state UI **Visual Style:** Futuristic + Premium Web3 + Data-Dense **Reusability Score:** 9/10

---

### 5. ZENTRY

**Link:** https://zentry.com **GitHub:** N/A **Type:** Web3 Gaming Landing Page **Why It's Relevant:** Zentry adalah **gold standard** untuk cinematic Web3 gaming landing page — scroll-triggered animations, 3D parallax, glowing elements, video background. Sangat cocok untuk landing/hero section "Last Man Standing" **Best Elements:** Cinematic hero section, scroll-triggered reveal, glowing card borders, premium typography (Zentry font), dark-to-purple gradient, video background overlay **Visual Style:** Cyberpunk + Cinematic + Premium **Reusability Score:** 8/10

---

## 🏆 TIER A — SANGAT RELEVAN

---

### 6. MAGIC UI

**Link:** https://magicui.design **GitHub:** https://github.com/magicuidesign/magicui ⭐ 20,781 **Type:** Open-Source Animated React UI Library **Why It's Relevant:** Library open-source TERBAIK untuk komponen yang dibutuhkan Last Man Standing. Punya komponen siap pakai: Animated List (cocok untuk live elimination feed), Number Ticker (cocok untuk countdown & prize pool counter), Marquee (untuk scrolling players), Border Beam (glow efek pada player card), Particles (background effect), Spotlight (highlight pada player yang menang) **Best Elements:**

- `AnimatedList` → Live Feed eliminasi
- `NumberTicker` → Countdown timer & prize pool realtime
- `BorderBeam` → Glow border hexagon player card
- `Particles` → Background arena
- `Marquee` → Scrolling player addresses

- `Ripple` → Efek saat eliminasi **Visual Style:** Modern + Dark + Animated **Reusability Score:** 10/10

---

## 7. ACETERNITY UI

**Link:** https://ui.aceternity.com/components **GitHub:** https://github.com/aceternity (200+ components) **Type:** Tailwind CSS + Framer Motion Component Library **Why It's Relevant:** Komponen premium dengan dark aesthetic yang langsung bisa digunakan. Punya: Glowing Cards (untuk player cards), Lamp Effect (untuk winner reveal), Aurora Background, Spotlight, Wavy Background, 3D Card Effect, Typewriter Effect **Best Elements:**

- `GlowingStarsBackgroundCard` → Alive player card
- `LampComponent` → Winner announcement overlay
- `BackgroundGradientAnimation` → Arena background
- `CardSpotlight` → Hover efek player card
- `WobbleCard` → Room card dengan physics **Visual Style:** Glassmorphism + Neon + Dark Premium **Reusability Score:** 9/10

---

## 8. FRAMER MOTION (Library)

**Link:** https://www.framer.com/motion/ **GitHub:** https://github.com/motiondivision/motion ⭐ 27,000+ **Type:** React Animation Library **Why It's Relevant:** Wajib digunakan untuk semua animasi di Last Man Standing — eliminasi player dengan `AnimatePresence`, countdown bounce, prize pool number flip, card enter/exit animation **Best Elements:** `AnimatePresence` untuk remove player dari grid, layout animations, spring physics, gesture animations, stagger children (reveal semua player saat game start) **Visual Style:** Any (library murni) **Reusability Score:** 10/10

---

## 9. SIVERT-IO / MATCHZY-AUTO-TOURNAMENT

**Link:** https://github.com/sivert-io/matchzy-auto-tournament ⭐ 26 **GitHub:** https://github.com/sivert-io/matchzy-auto-tournament **Type:** Esports Tournament Realtime Dashboard **Why It's Relevant:** Open-source CS2 tournament tracker dengan React + WebSocket + TypeScript — persis stack yang dibutuhkan. Punya: real-time match tracking, bracket system, WebSocket integration, server allocation. Bisa dipelajari arsitekturnya untuk WebSocket-based game state management **Best Elements:** WebSocket real-time match state, tournament bracket UI, live match tracking, TypeScript React architecture **Visual Style:** Esports Dark **Reusability Score:** 8/10

---

## 10. 0XAXON7 / CASINO SMART CONTRACT WEB3 GAME

**Link:** https://github.com/0xAxon7/Casino_Smart_Contract_Web3_Game ⭐ 105 **GitHub:** https://github.com/0xAxon7/Casino_Smart_Contract_Web3_Game **Type:** Web3 On-Chain Casino Game (Solana/EVM/Monad) **Why It's Relevant: REPO PALING RELEVAN di GitHub untuk projek kamu.** Ini adalah on-chain casino jackpot game di Monad/Solana dengan: multiple players deposit → 1 winner takes all, smart contract vault, React + Tailwind frontend, provably fair VRF randomness. Jackpot mechanic-nya identik dengan Last Man Standing **Best Elements:** Jackpot round smart contract, join round UI, winner claim flow, React frontend architecture, Monad compatibility **Visual Style:** Web3 Casino Dark **Reusability Score:** 9/10

---

### 11. SAFIN-RWEB / ESCRIMS

**Link:** https://github.com/SafinRweb/Escrims **GitHub:** https://github.com/SafinRweb/Escrims **Type:** Esports Tournament Platform (React + TypeScript + Firebase) **Why It's Relevant:** Full-stack esports tournament platform open-source dengan: tournament brackets, live standings, real-time updates via Firebase, stream integration. Bisa diadaptasi untuk room management + live standings Last Man Standing **Best Elements:** Tournament bracket UI, live standings panel, stream embed integration, Firebase realtime database pattern, team/player management **Visual Style:** Modern Esports Dark **Reusability Score:** 7/10

---

## 🎯 TIER B — REFERENSI PENTING

---

### 12. SHADCN/UI

**Link:** https://ui.shadcn.com **GitHub:** https://github.com/shadcn-ui/ui ⭐ 80,000+ **Type:** Headless React Component Library **Why It's Relevant:** Base component library yang wajib digunakan — Card, Badge, Button, Progress, Separator, Tooltip semua tersedia. Dikombinasikan dengan dark theme dan custom neon color, ini akan jadi fondasi UI projek kamu **Best Elements:** Card component (room cards), Badge (LIVE NOW / WAITING status), Progress bar (player countdown), Table (leaderboard), Dialog (join confirmation) **Visual Style:** Minimal Dark (customizable) **Reusability Score:** 9/10

---

### 13. ALOTT2223 / CYBER CONTAINER PLATFORM

**Link:** https://github.com/alott2223/cyber-container-platform ⭐ 9 **GitHub:** https://github.com/alott2223/cyber-container-platform **Type:** Cyberpunk Dashboard (Go + React + WebSocket) **Why It's Relevant:** Platform dengan cyberpunk aesthetic + real-time WebSocket metrics + React/Next.js frontend. Arsitektur WebSocket Hub-nya (Go backend) bisa jadi referensi untuk real-time elimination event broadcasting **Best Elements:** Cyberpunk dark theme, real-time metrics dashboard, WebSocket hub architecture, live container status feed (mirip live elimination feed), terminal integration **Visual Style:** Cyberpunk Dark + Neon **Reusability Score:** 7/10

---

## 14. VALORANT ESPORTS UI (Riot Games)

**Link:** https://valorantesports.com **GitHub:** N/A **Type:** Esports Spectator Interface **Why It's Relevant:** Referensi terbaik untuk spectator mode design — live score overlay, player HUD, kill feed (inspirasi untuk elimination feed), minimap (bisa diadaptasi jadi arena grid), agent portrait cards **Best Elements:** Kill/elimination feed di sudut kiri, floating player health bars, round timer di tengah atas, minimap grid, spectator camera controls, round win/lose animation **Visual Style:** Futuristic Esports + Military HUD **Reusability Score:** 8/10

---

## 15. THE FINALS (Game UI)

**Link:** https://www.reachthefinals.com **GitHub:** N/A **Type:** Battle Royale Game UI Reference **Why It's Relevant:** The Finals punya elimination system UI yang identik dengan konsep Last Man Standing — realtime player count ("ALIVE: 24"), team elimination popup, cash-out timer (mirip elimination countdown), leaderboard panel **Best Elements:** "X Players Remaining" HUD overlay, elimination notification popup, cash-out/round timer, leaderboard side panel, team color coding **Visual Style:** Gritty Dark + Orange Accent + Cinematic **Reusability Score:** 8/10

---

## 16. APEX LEGENDS HUD

**Link:** https://www.ea.com/games/apex-legends (reference only) **GitHub:** N/A **Type:** Battle Royale HUD Reference **Why It's Relevant:** Apex Legends punya squad status panel yang mirip dengan player arena grid-mu — hexagonal/square player indicators, alive/eliminated status, shield indicator (Shield Ready badge-mu persis mirip ini), ring countdown **Best Elements:** Squad status panel (mirip arena grid kamu), shield/armor indicator, legend portrait cards, ring/zone countdown timer, kill feed di pojok kanan **Visual Style:** Sci-Fi + Dark + Neon Orange **Reusability Score:** 8/10

---

## 17. TENSOR TRADE

**Link:** https://www.tensor.trade **GitHub:** N/A **Type:** Solana NFT Trading Dashboard **Why It's Relevant:** Tensor adalah contoh terbaik untuk "premium crypto-native dark UI" di Solana ecosystem — realtime price updates, wallet integration UI, activity feed (mirip live feed eliminasi), collection grid (mirip player arena grid) **Best Elements:** Realtime activity feed sidebar, dark premium aesthetic, wallet connect button style, collection/player grid layout, live transaction feed **Visual Style:** Premium Dark + Solana Purple **Reusability Score:** 7/10

---

## 18. KAMINO FINANCE

**Link:** https://kamino.finance **GitHub:** N/A **Type:** Solana DeFi Dashboard **Why It's Relevant:** Kamino punya salah satu UI DeFi terbagus di Solana — progress bar prize pool (mirip TVL bar), stats cards (mirip Round/Alive/Eliminated stats bar kamu), clean dark typography **Best Elements:** Stats overview bar (Room/Round/Alive/Prize Pool mirip layout kamu), progress indicators, glow

pada angka penting, wallet portfolio view, dark purple accent **Visual Style:** Premium DeFi Dark + Purple Gradient **Reusability Score:** 7/10

---

# 💻 TIER C — GITHUB REPOS & UI KITS

---

## 19. GGPIO18 / REACT-CYBERPUNK-BENTO-V2-DASHBOARD

**Link:** https://github.com/ggpio18/REACT-CYBERPUNK-BENTO-V2-DASHBOARD-PROJ13- **GitHub:** https://github.com/ggpio18/REACT-CYBERPUNK-BENTO-V2-DASHBOARD-PROJ13- **Type:** Cyberpunk Bento Grid Dashboard (React) **Why It's Relevant:** Template dashboard cyberpunk dengan Bento grid layout — bisa diadaptasi untuk "stats overview" section di arena view (Round, Alive, Eliminated, Prize Pool, Countdown) **Best Elements:** Bento grid layout, cyberpunk color palette, neon borders, dark card components **Visual Style:** Cyberpunk Neon **Reusability Score:** 6/10

---

## 20. EGGPLANT203 / INFINITE-TIC-TAC-TOE

**Link:** https://github.com/Eggplant203/Infinite-Tic-Tac-Toe **GitHub:** https://github.com/Eggplant203/Infinite-Tic-Tac-Toe **Type:** Realtime Multiplayer Game (React + Socket.IO + Framer Motion) **Why It's Relevant:** Open-source realtime multiplayer game dengan stack persis yang kamu butuhkan: React + TypeScript + Socket.IO + Framer Motion. Bisa dijadikan referensi arsitektur untuk multiplayer game state management **Best Elements:** Socket.IO room management, real-time game state sync, Framer Motion animations, responsive game board layout, player join/leave handling **Visual Style:** Modern Dark **Reusability Score:** 7/10

---

## 21. NEKOTHEDEV / PLANIT (Cyberpunk 2077-Inspired)

**Link:** https://github.com/NekoTheDev/PlanIt **GitHub:** https://github.com/NekoTheDev/PlanIt **Type:** Cyberpunk Dashboard (TypeScript + React) **Why It's Relevant:** PlanIt 2077 adalah cyberpunk-inspired dashboard dengan TypeScript dan MSW (Mock Service Worker) — bisa dijadikan referensi untuk cyberpunk color palette, typography, dan data visualization style **Best Elements:** Cyberpunk 2077 color palette (yellow-on-black), kanban-style card management, timeline views, responsive dark layout **Visual Style:** Cyberpunk 2077 (Yellow + Dark) **Reusability Score:** 6/10

---

## 22. ADITYA-2129 / V-RIVELS (Valorant Tournament)

**Link:** https://github.com/aditya-2129/V-Rivels **GitHub:** https://github.com/aditya-2129/V-Rivels **Type:** Valorant Esports Tournament Platform (Next.js 15) **Why It's Relevant:** Premium Valorant tournament platform dengan: real-time brackets, automated stat syncing, team discovery, Next.js 15

+ Tailwind CSS + Appwrite. Sangat mirip arsitektur yang dibutuhkan Last Man Standing **Best Elements:** Real-time tournament brackets, player stat cards, team finder UI, live match tracking, Next.js 15 architecture **Visual Style:** Premium Esports (Valorant-inspired) **Reusability Score:** 7/10

---

# 🌐 REFERENSI WEBSITE / LANDING PAGE

---

### 23. BLAST.IO

**Link:** https://blast.io **GitHub:** N/A **Type:** Web3 Gaming Ecosystem Landing Page **Why It's Relevant:** Blast adalah L2 gaming-focused chain dengan landing page yang menampilkan game cards, prize pools, live players — hampir identik dengan lobby grid Last Man Standing **Best Elements:** Game room cards, live prize pool counters, player count badges, dark neon aesthetic, animated particle background **Visual Style:** Neon Dark + Web3 Gaming **Reusability Score:** 8/10

---

### 24. SUPERFLUID FINANCE

**Link:** https://superfluid.finance **GitHub:** https://github.com/superfluid-finance **Type:** Realtime Streaming Payments Dashboard **Why It's Relevant:** Superfluid punya UI untuk "streaming money in real-time" — progress bar yang terus berjalan (mirip countdown eliminasi), live number counter, realtime transaction feed. Visual language-nya sangat cocok untuk "prize pool streaming to winner" **Best Elements:** Real-time streaming progress bars, live number counters, transaction flow animation, wallet-centric UI **Visual Style:** Modern Dark + Stream Animation **Reusability Score:** 6/10

---

### 25. NANSEN.AI PORTFOLIO

**Link:** https://portfolio.nansen.ai **GitHub:** N/A **Type:** Crypto Analytics Dashboard **Why It's Relevant:** Nansen punya salah satu UI analytics crypto terbaik — realtime wallet tracking (mirip tracking player wallets), activity feed (mirip live elimination feed), dark premium aesthetic dengan purple/teal accent **Best Elements:** Wallet activity feed, real-time transaction events, player/wallet avatar grid, premium dark purple theme **Visual Style:** Premium Analytics Dark + Purple **Reusability Score:** 6/10

---

# 🛠️ LIBRARY & TOOLS WAJIB

---

### 26. TAILWIND CSS + TAILWIND VARIANTS

**Link:** https://tailwindcss.com / https://www.tailwind-variants.org **GitHub:** https://github.com/tailwindlabs/tailwindcss ⭐ 84,000+ **Type:** CSS Framework **Why It's Relevant:** Wajib untuk rapid development hackathon. Custom config untuk warna neon (purple `#7C3AED`, teal `#14F195`, orange `#F59E0B`) sesuai brand Last Man Standing **Best Elements:** Utility classes, dark mode, animation classes, JIT compiler, custom color config **Reusability Score:** 10/10

---

## 27. RECHARTS / CHART.JS

**Link:** https://recharts.org **GitHub:** https://github.com/recharts/recharts ⭐ 24,000+ **Type:** React Charting Library **Why It's Relevant:** Untuk visualisasi prize pool growth chart, elimination rate per round, player count timeline — bisa membuat arena view lebih data-rich **Best Elements:** Area charts, animated line charts, realtime data updates, dark theme support **Reusability Score:** 7/10

---

## 28. SOCKET.IO CLIENT

**Link:** https://socket.io **GitHub:** https://github.com/socketio/socket.io ⭐ 61,000+ **Type:** WebSocket Library **Why It's Relevant:** Wajib untuk realtime elimination events, countdown sync, player join/leave, live feed updates **Best Elements:** Room management, event broadcasting, reconnection handling, TypeScript support **Reusability Score:** 10/10

---

## 29. VIEM + WAGMI

**Link:** https://viem.sh / https://wagmi.sh **GitHub:** https://github.com/wevm/viem ⭐ 2,800+ | https://github.com/wevm/wagmi ⭐ 5,900+ **Type:** Web3 React Hooks Library **Why It's Relevant:** Stack standar untuk wallet integration di Monad — Connect Wallet button, read on-chain prize pool, watch contract events (eliminasi on-chain) **Best Elements:** `useAccount`, `useBalance`, `useWatchContractEvent` (untuk listen eliminasi on-chain), `useWriteContract` (join room) **Reusability Score:** 10/10

---

## 30. RAINBOWKIT

**Link:** https://www.rainbowkit.com **GitHub:** https://github.com/rainbow-me/rainbowkit ⭐ 4,500+ **Type:** Wallet Connect UI Library **Why It's Relevant:** "Connect Wallet" button yang sudah siap pakai dengan UI premium. Monad compatible, support MetaMask/Coinbase/WalletConnect. Satu-satunya pilihan untuk hackathon **Best Elements:** Animated connect button, multi-wallet modal, chain switcher, dark theme built-in **Reusability Score:** 9/10

Berikut 6 referensi website Web3 GameFi dengan konsep arena / last-man-standing / lobby PvP yang mirip dengan gambar 1 & 2 kamu (room cards + player arena grid + live feed):

**1. Sybils (sybils.app) — Monad Testnet** https://sybils.app Paling dekat dengan konsep "LAST MAN STANDING" kamu — bahkan disebut eksplisit sebagai codename campaign-nya di halaman utama. Sama-sama jalan di Monad Testnet. UI-nya dark/neon cyberpunk, ada wallet connect, tampilan mint/room campaign. Bagus untuk inspirasi branding "powered by Monad" + tipografi dan warna neon yang kamu pakai.

**2. Overworld Arena** https://overworld.gg (artikel review: https://games.gg/news/overworld-arena-web3-pfp-battler-with-big-rewards/) Web3 PFP battler dengan prize pool besar ($100K+ USDT). UI-nya punya tampilan arena, leaderboard, entry fee, dan player profile card — sangat mirip dengan struktur informasi di card room kamu (Prize Pool, Entry, Players).

**3. Blast Royale (NOOB)** https://blastroyale.com Web3 mobile + web battle royale dengan NFT. Punya lobby dengan entry fee, tournament bracket, prize pool display, dan live match feed. Tampilan futuristik dengan panel-panel glow — cocok untuk referensi bagaimana menampilkan "player arena grid" dengan avatar/NFT.

**4. Supersize (supersize.gg)** Realtime multiplayer on-chain action game (disebut di thegrid.id). Pemain bayar entry fee, collect tokens, exit untuk withdraw. UI lobby-nya sederhana tapi punya live feed elimination dan prize pool real-time — persis seperti panel "Live Feed" di gambar 2 kamu.

**5. Knockout (GitHub open-source repo) — REPO YANG KAMU MINTA** https://github.com/dkwhitedevil/Knockout Ini jackpot buat kamu. Card battle royale, 4–8 player, entry fee (5 USDC), round 10–15 detik, eliminasi per HP, last man standing wins pot. Built dengan **Next.js + TypeScript** (frontend folder siap di-clone). Konsep gameplay-nya hampir identik dengan milikmu — kamu bisa belajar struktur komponen room, round timer, elimination log, dan settlement flow langsung dari source code-nya.

**6. Off The Grid (Gunzilla Games)** https://www.offthegrid.game Battle royale AAA on-chain (GUNZ chain). Meski game 3D, website/lobby web-nya punya UI keren: room browser, match queue, stats panel, dan live roster. Bagus untuk inspirasi visual hierarchy, cinematic dark theme, dan bagaimana menampilkan "alive / eliminated" status secara visual.

---

**Bonus tip untuk UI kamu:**

- Untuk eksplorasi visual murni (bukan live dapp), cek **Dribbble → tag "gamefi"** (https://dribbble.com/tags/gamefi) dan cari kata kunci "battle royale dashboard", "tournament lobby web3", "arena UI". Banyak mockup Figma yang bisa kamu adopsi pattern card/grid-nya.
- Untuk komponen arena grid hexagon seperti di gambar 2, coba lihat juga **Dark Forest** (zkga.me) — game on-chain dengan grid map yang iconic.

Repo **Knockout** (poin 5) paling worth ditelusuri kalau kamu mau langsung lihat implementasi kode React/Next.js-nya sebagai fondasi UI kamu.

# Building

**Smart Contract**
explorer monad: https://testnet.monadvision.com/
RPC_URL=https://testnet-rpc.monad.xyz

**Chain Monad**


```
{
  "naainId": 10143,
  "rpcUrl": "https://testnet-rpc.monad.xyz",
  "explorer": "https://testnet.monadexplorer.com",
  "nativeCurrency": {
    "name": "MON",
    "symbol": "MON",
    "decimals": 18
  }
}me": "Monad Testnet",
  "ch
```

Tab 8